Smart City Traffic Analytics

Background

The City of Metropolia operates 1,800 roadside sensor units and 220 traffic cameras across major corridors. City leaders want near real-time visibility into congestion and incidents, and a historical store for planning, policy, and environmental reporting.

Business Objectives

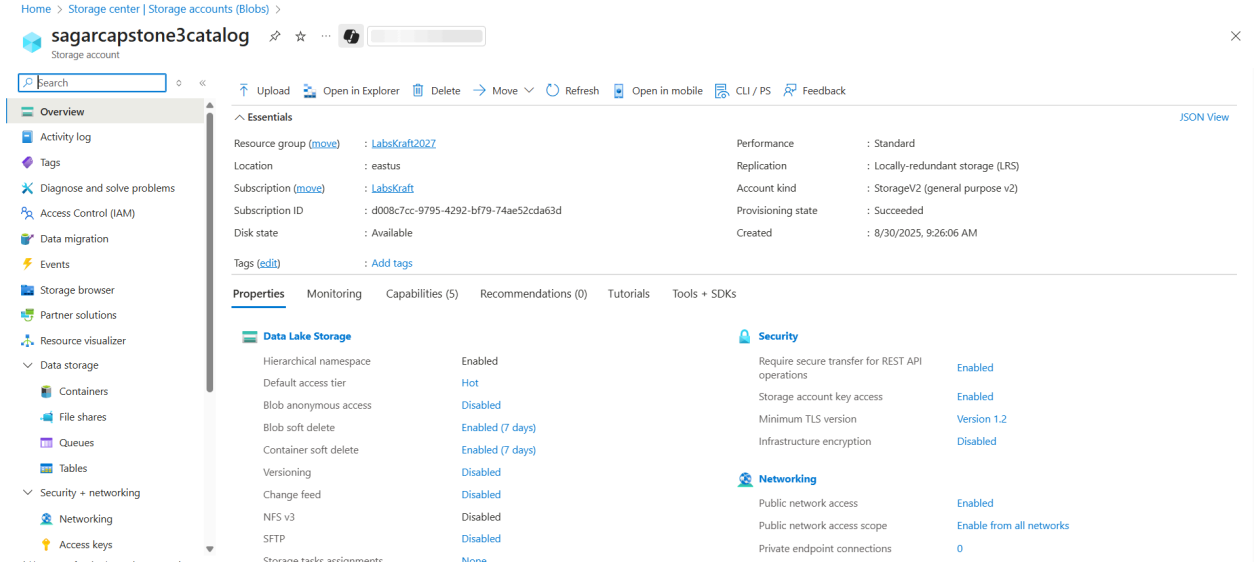
1. Provide live congestion status per intersection and corridor with alerts for abnormal conditions (e.g., sudden volume drop, severe slowdown, blocked lane).

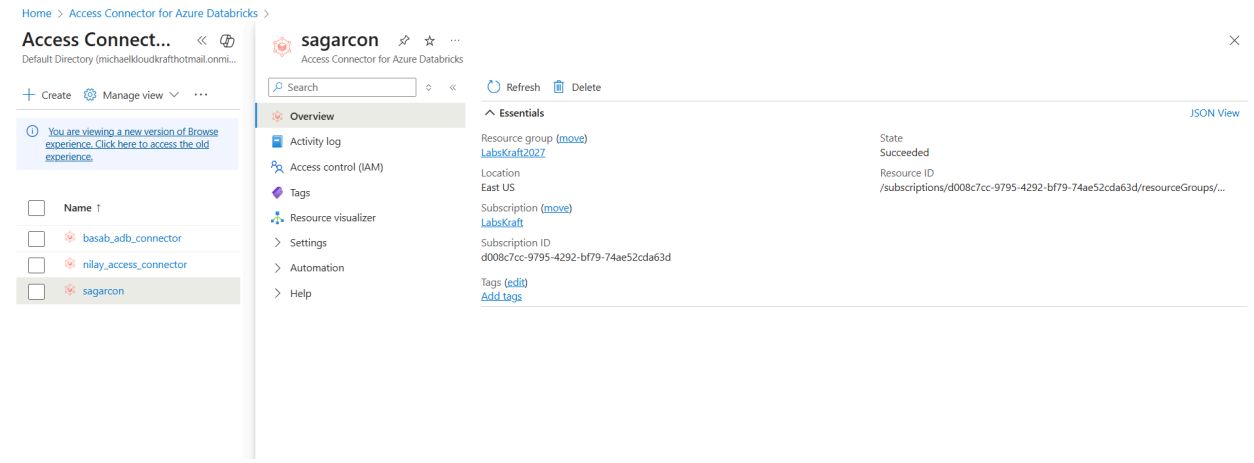
2. Quantify the impact of congestion on air quality near schools/hospitals and identify high-risk windows by time of day and season.

3. Produce 15–30 minute congestion forecasts for top 50 intersections to assist signal timing and route advisories.

Supply a governed, self-service analytics layer for planners and external BI tools, including auditable historical data.

Creation of external location for creation of our unity catalog:





Home > Storage center | Storage accounts (Blobs) > sagarcapstone3catalog | Access Control (IAM) >

Add role assignment ...

Role | Members | Conditions | Review + assign

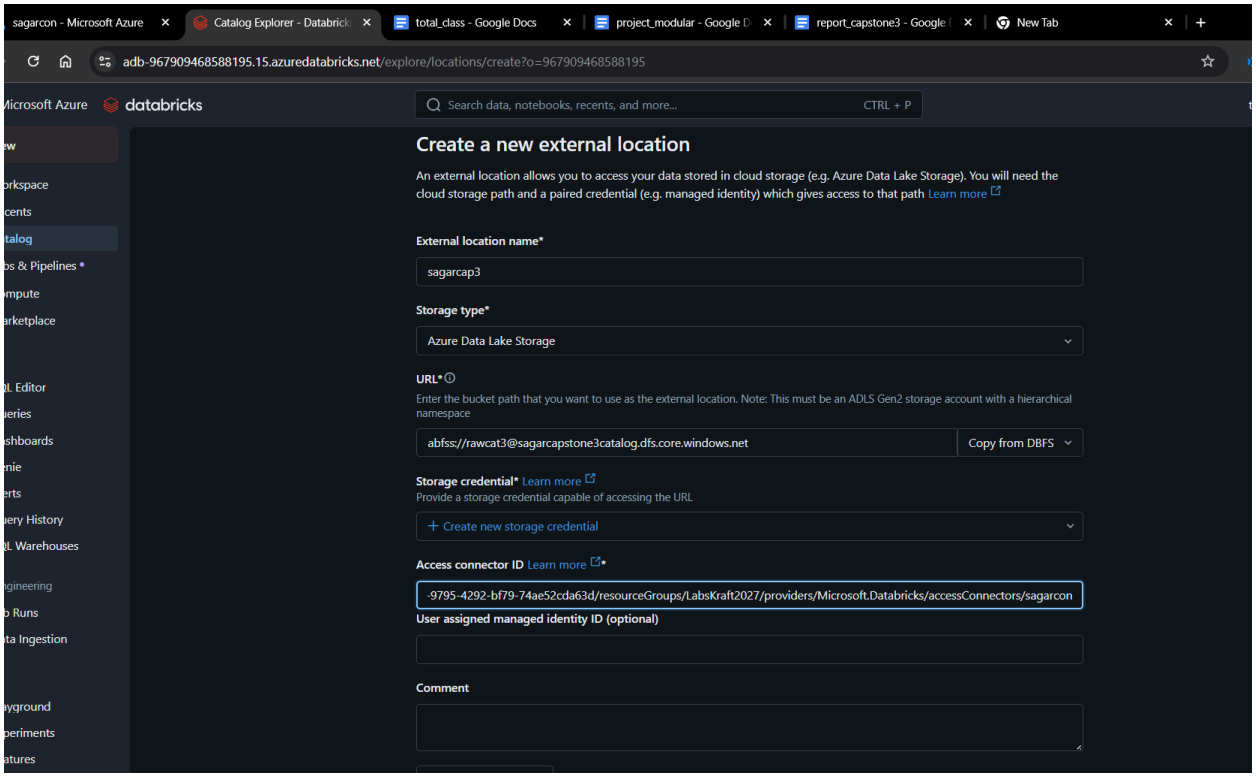
Role	Storage Blob Data Contributor
Scope	/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/LabsKraft2027/providers/Microsoft.Storage/storageAccounts/sagarcapstone3catalog

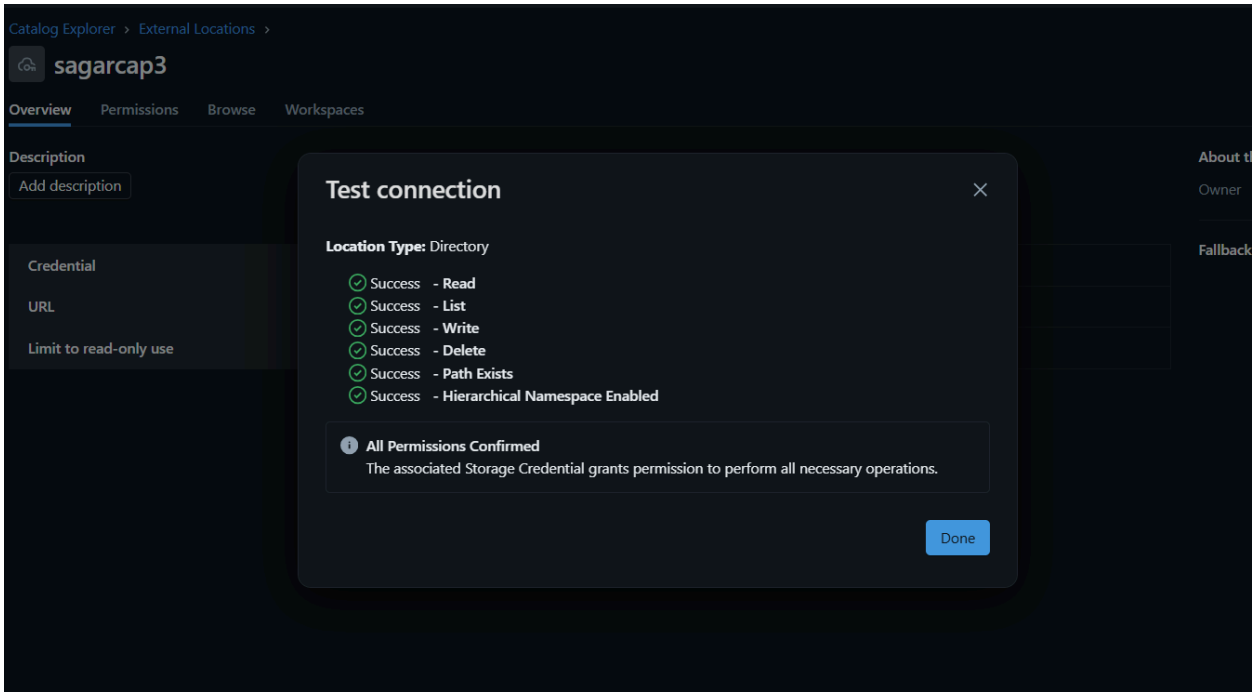
Members

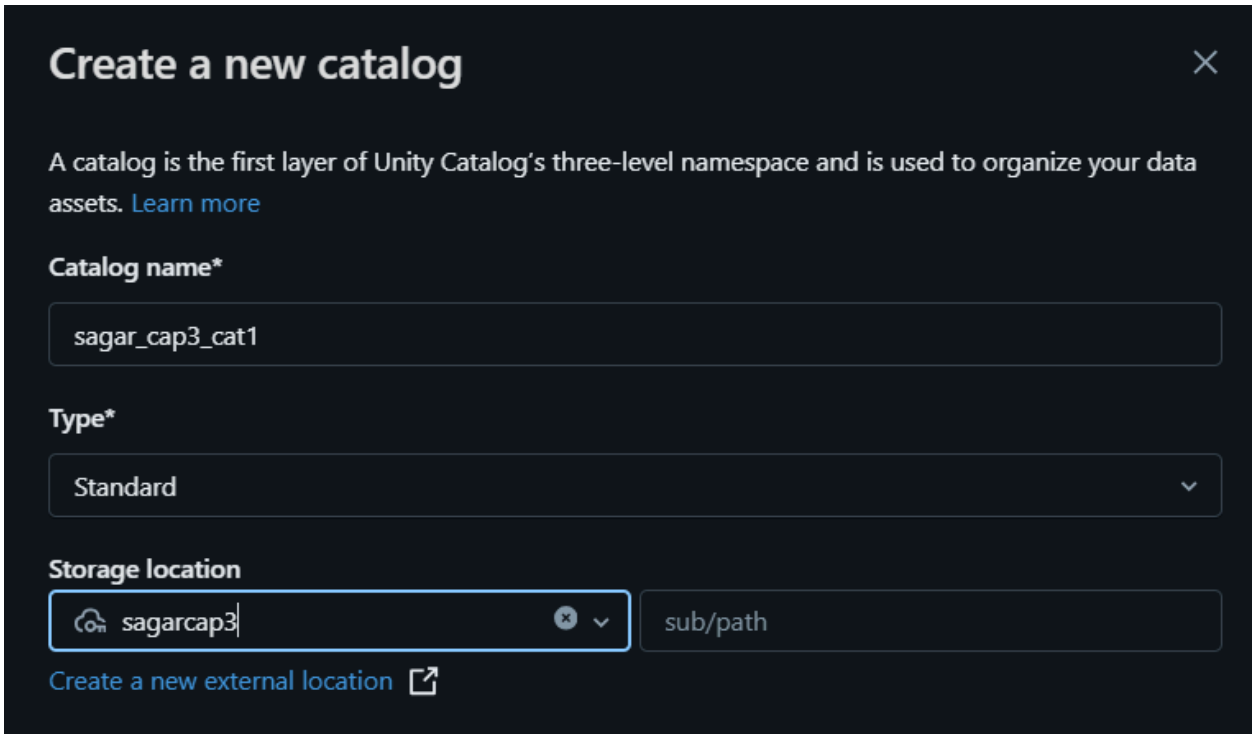
Name	Object ID	Type
sagarcon	e6bc38b0-b354-42a1-9dab-54b5874b2283	Access Connector for Azure Databricks

Description	No description
Condition	None

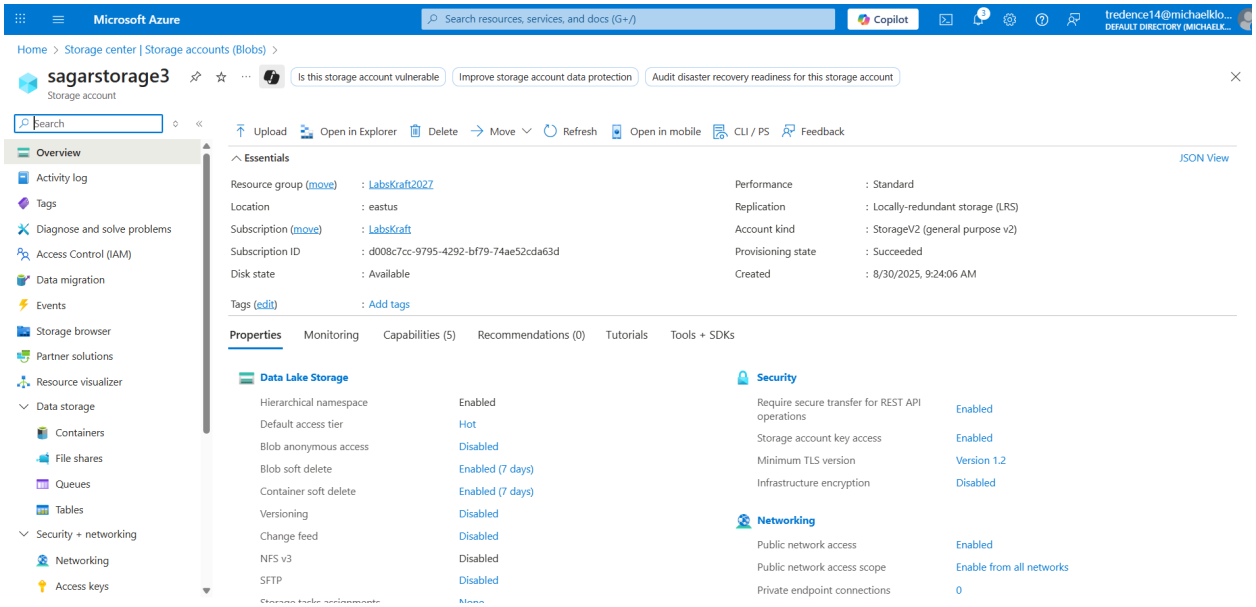
Review + assign | Previous | Next




Catalog Explorer › External Locations ›
sagarcap3
Overview
Permissions
Browse
Workspaces
Description
Add description
Credential
URL
Limit to read-only use
Test connection
Location Type: Directory
Success - Read
Success - List
Success - Write
Success - Delete
Success - Path Exists
Success - Hierarchical Namespace Enabled
All Permissions Confirmed
The associated Storage Credential grants permission to perform all necessary operations.
Done
About t
Owner
Fallback


Create a new catalog
A catalog is the first layer of Unity Catalog's three-level namespace and is used to organize your data assets. Learn more
Catalog name*
sagar_cap3_cat1
Type*
Standard
Storage location
sagarcap3
sub/path
Create a new external location

Creation of storage for storing other non-streaming files:



✅ Python program to send files to adls:

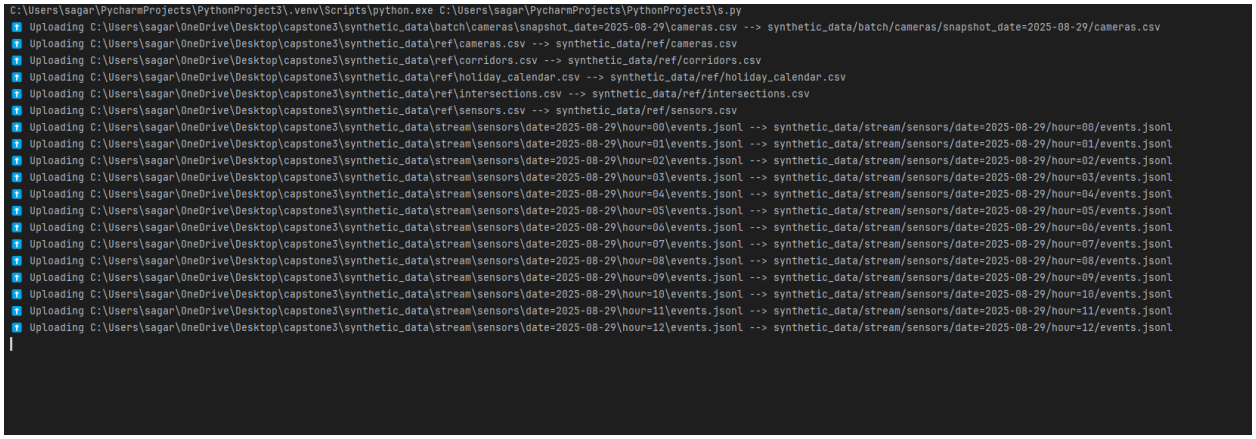
- Connects to your Azure Data Lake container `raw33`.
- Walks through all files in your local folder `capstone3`.
- Uploads each file to ADLS, keeping the same folder structure.
- Prints progress logs and shows if something fails.

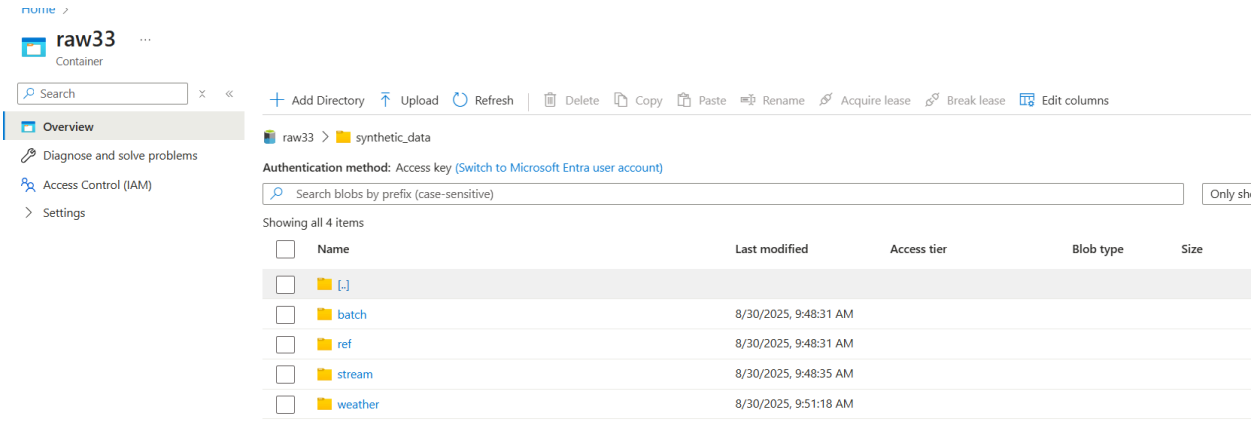
installation of some libraries using windows powershell:
python -m pip install --upgrade pip
python -m pip install azure-storage-file-datalake

Program for sending all the files from my local folder to ADLS: s.py

```
import os
from azure.storage.filedatalake import DataLakeServiceClient

# Connection string
connect_str =
"DefaultEndpointsProtocol=https;AccountName=sagarstorage3;Accoun
tKey=aYUdqF7AzagGq+0dw06s7i8wd5rBXjFpMnxNIrXDZxkOWkFMz6oOR0JxwNb
ItLvsgNSUjsuMKEcD+ASt+OAkKw==;EndpointSuffix=core.windows.net"

# Container (filesystem) name
container_name = "raw33"

# Local folder to upload
local_folder_path = r"C:\Users\sagar\OneDrive\Desktop\capstone3"

try:
   # Create DataLake client
   service_client =
DataLakeServiceClient.from_connection_string(connect_str)
   file_system_client =
service_client.get_file_system_client(file_system=container_name
)

   # Walk through all files in local folder
   for root, dirs, files in os.walk(local_folder_path):
       for file_name in files:
           # Full local file path
           local_file_path = os.path.join(root, file_name)

           # Create relative path for ADLS (remove base folder
from path)
           relative_path = os.path.relpath(local_file_path,
local_folder_path)
```

```
            adls_path = f"{relative_path}".replace("\\", "/")

            # Get file client
            file_client =
file_system_client.get_file_client(adls_path)

            print(f"⬆️ Uploading {local_file_path} -->
{adls_path}")

            # Upload file
            with open(local_file_path, "rb") as data:
                file_client.upload_data(data, overwrite=True)

    print("✅ All files uploaded successfully!")

except Exception as ex:
    print("❌ Exception:")
    print(ex)
```





weather directory has many files per zone combining to 200 rows almost . It's better to merge and get single file and we have done this in bronze layer.

Bronze Layer

```
%sql
use catalog sagar_cap3_cat1;
create schema if not exists ref;
create schema if not exists weather
```

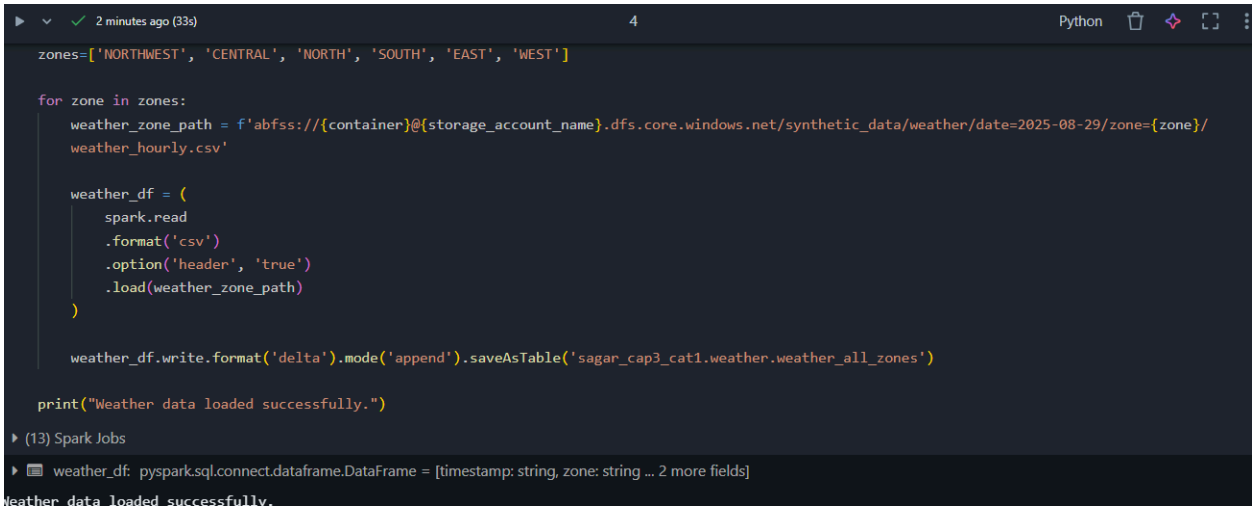
ADLS ingestion:

```
# Set the Azure storage account credentials securely
storage_account_name = 'sagarstorage3'
storage_account_key='aYUdqF7AzagGq+0dw06s7i8wd5rBXjFpMnxNIrXDZxkOWkFMz6oOR0JxwNbItLvsgNSUjsuMKEcD+ASt+OAkKw=='
container='raw33'
spark.conf.set(
```

```
f"fs.azure.account.key.{storage_account_name}.dfs.core.windows.net",
    storage_account_key
)

# Loading files from ref folder from adls:
file_names = ['cameras', 'corridors', 'holiday_calendar', 'intersections', 'sensors']
for file_name in file_names:
    print(f"Loading ref file: {file_name}.csv")
    ref_df = (
        spark.read
        .format("csv")
        .option("header", "true")

.load(f'abfss://{container}@{storage_account_name}.dfs.core.windows.net/synthetic_data/ref/{file_name}.csv')
    )

ref_df.write.format('delta').mode('overwrite').saveAsTable(f'sagar_cap3_cat1.ref.{file_name}')

print("Reference data loaded successfully.")
```

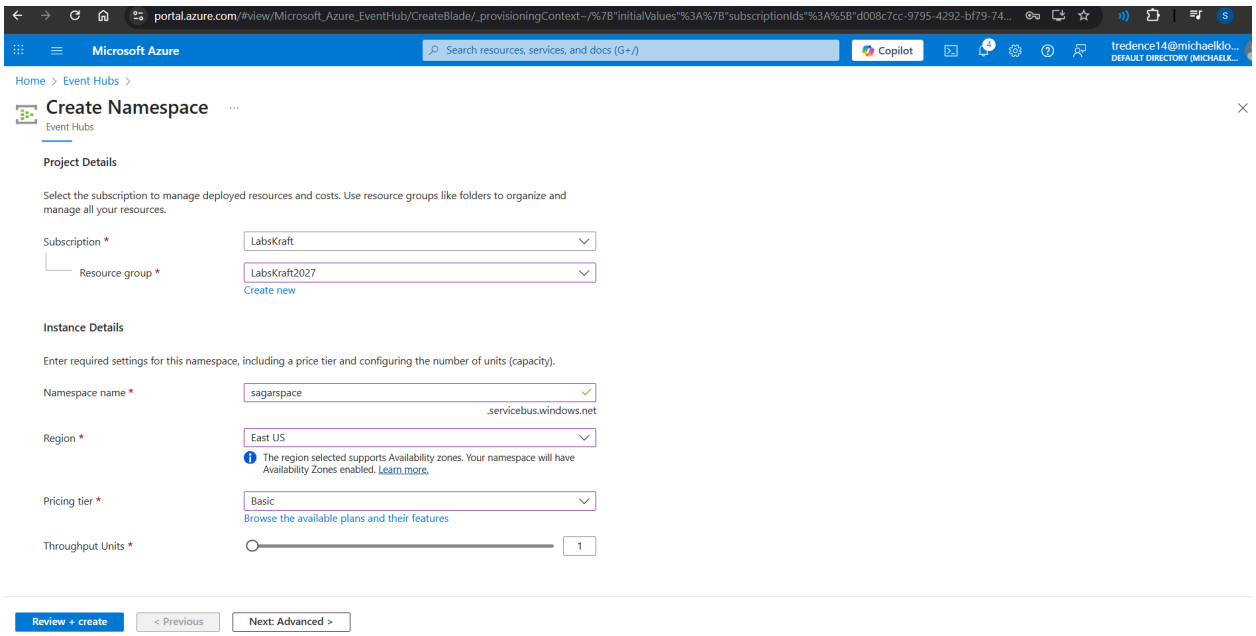
```
file_names = ['cameras', 'corridors', 'holiday_calendar', 'intersections', 'sensors']
for file_name in file_names:
    print(f"Loading ref file: {file_name}.csv")
    ref_df = (
        spark.read
        .format("csv")
        .option("header", "true")
        .load(f'abfss://{container}@{storage_account_name}.dfs.core.windows.net/synthetic_data/ref/{file_name}.csv')
    )
    ref_df.write.format('delta').mode('overwrite').saveAsTable(f'sagar_cap3_cat1.ref.{file_name}')

print("Reference data loaded successfully.")


(14) Spark Jobs
Loading ref file: cameras.csv
Loading ref file: corridors.csv
Loading ref file: holiday_calendar.csv
Loading ref file: intersections.csv
Loading ref file: sensors.csv
Reference data loaded successfully.
```

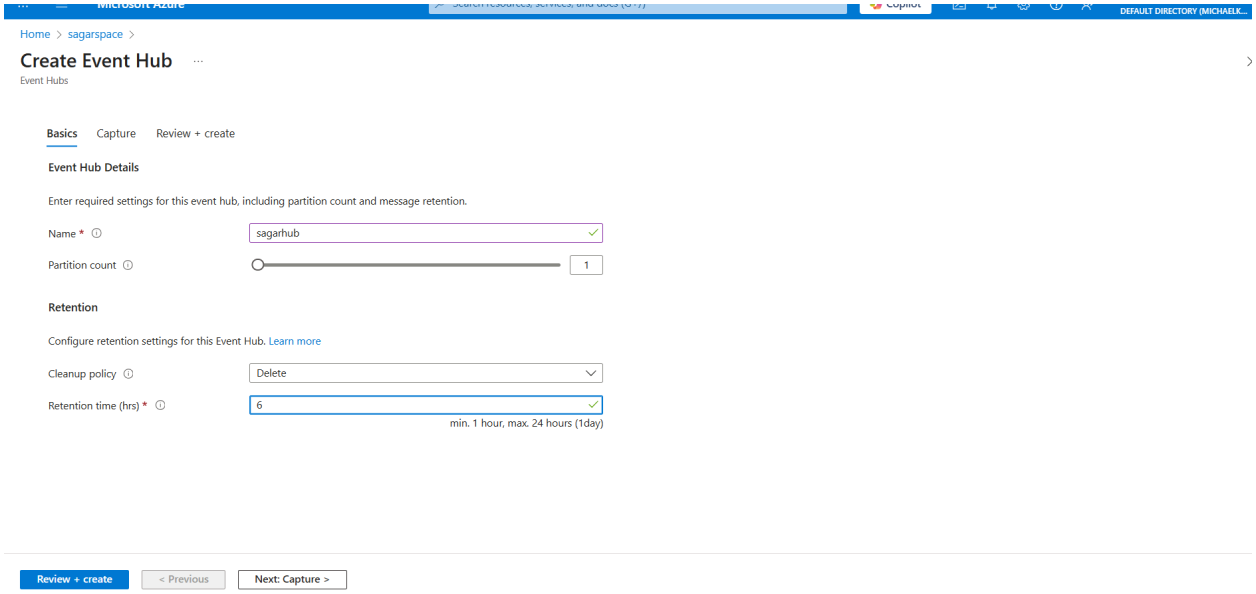
```
# Loading weather_hourly files for all zones from weather folder
zones=['NORTHWEST', 'CENTRAL', 'NORTH', 'SOUTH', 'EAST', 'WEST']

for zone in zones:
    weather_zone_path = f'abfss://{container}@{storage_account_name}.dfs.core.windows.net/synthetic_data/weather/date=2025-08-29/zone={zone}/weather_hourly.csv'

    weather_df = (
        spark.read
        .format('csv')
        .option('header', 'true')
        .load(weather_zone_path)
    )


weather_df.write.format('delta').mode('append').saveAsTable('sagar_cap3_cat1.weather.weather_all_zones')

print("Weather data loaded successfully.")
```

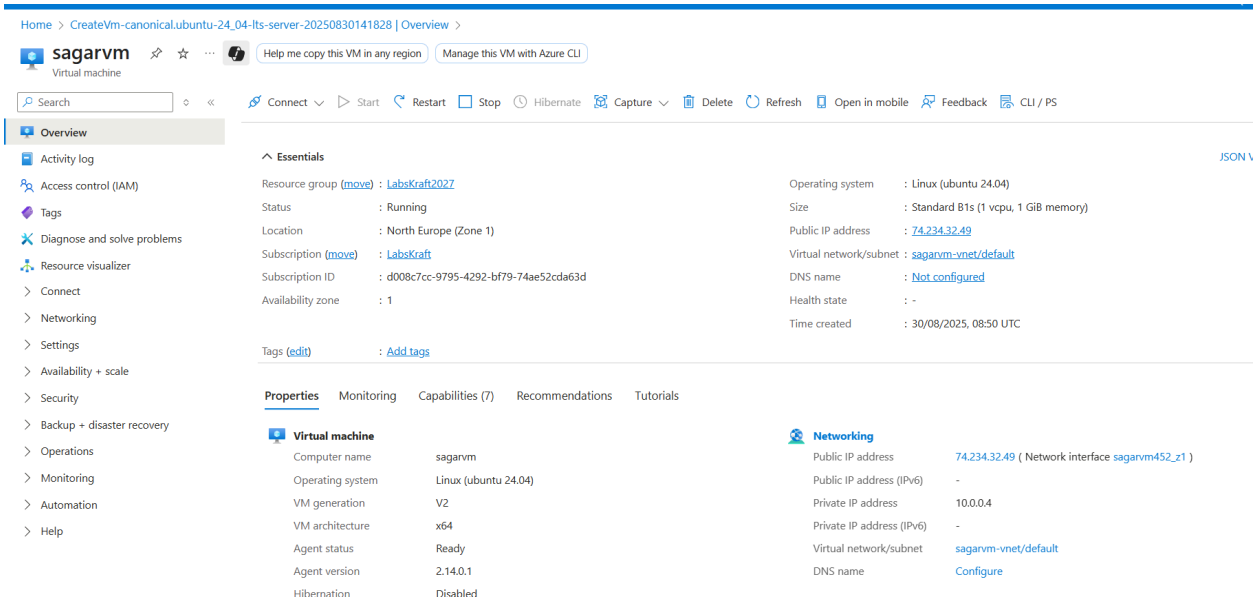


Event Hubs to Delta Streaming in Databricks:

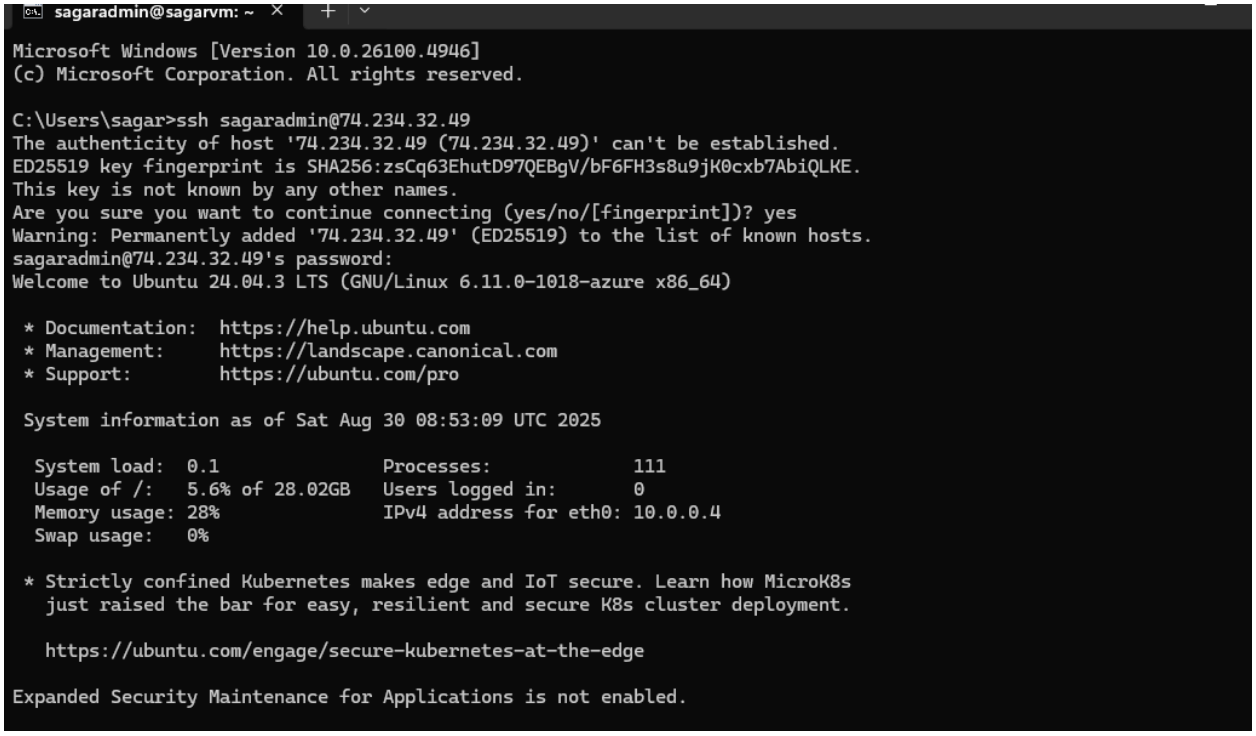
creation of namespace and eventhub:



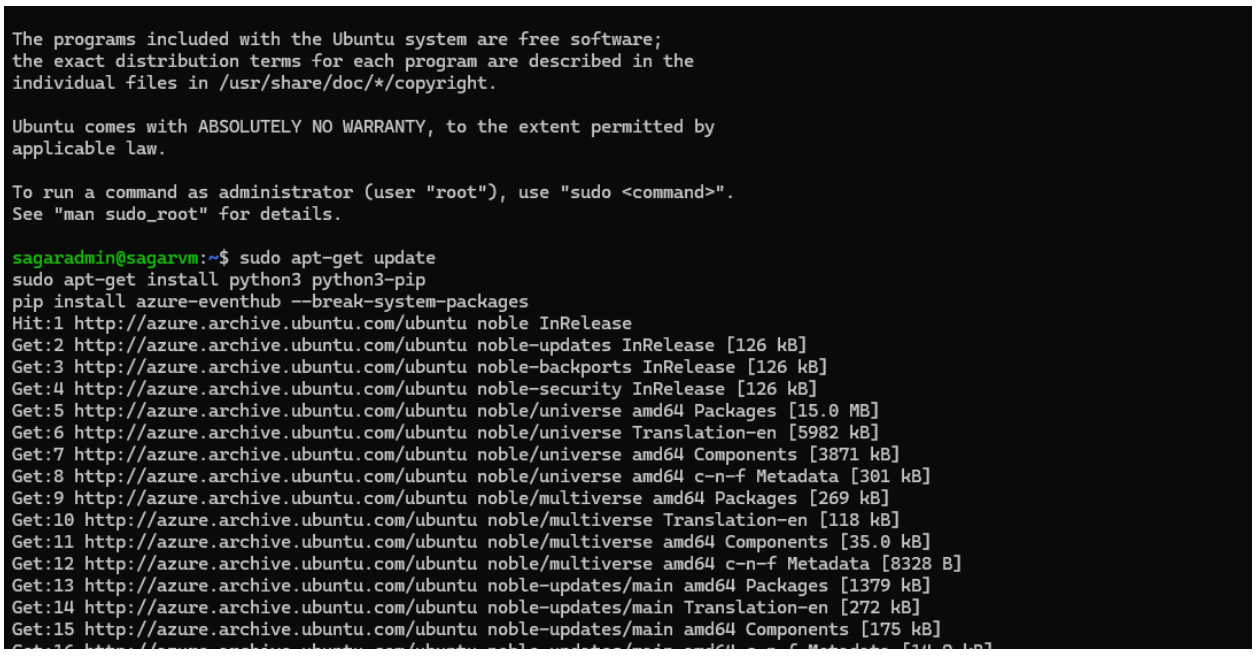

Home > sagarspace >
Create Event Hub
Event Hubs
Basics
Capture
Review + create
Event Hub Details
Enter required settings for this event hub, including partition count and message retention.
Name *
sagarhub
Partition count
1
Retention
Configure retention settings for this Event Hub. Learn more
Cleanup policy
Delete
Retention time (hrs) *
6
min. 1 hour, max. 24 hours (1day)
Review + create
< Previous
Next: Capture >


Home > CreateVm-canonical.ubuntu-24_04-lts-server-20250830141828 | Overview >
sagarvm
Virtual machine
Help me copy this VM in any region
Manage this VM with Azure CLI
Search
Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Resource visualizer
Connect
Networking
Settings
Availability + scale
Security
Backup + disaster recovery
Operations
Monitoring
Automation
Help
Connect
Start
Restart
Stop
Hibernate
Capture
Delete
Refresh
Open in mobile
Feedback
CLI / PS
Essentials
JSON
Resource group (move) : LabsKraft2027
Status : Running
Location : North Europe (Zone 1)
Subscription (move) : LabsKraft
Subscription ID : d008c7cc-9795-4292-bf79-74ae52cda63d
Availability zone : 1
Operating system : Linux (ubuntu 24.04)
Size : Standard B1s (1 vcpu, 1 GiB memory)
Public IP address : 74.234.32.49
Virtual network/subnet : sagarvm-vnet/default
DNS name : Not configured
Health state : -
Time created : 30/08/2025, 08:50 UTC
Tags (edit) : Add tags
Properties
Monitoring
Capabilities (7)
Recommendations
Tutorials
Virtual machine
Computer name sagarvm
Operating system Linux (ubuntu 24.04)
VM generation V2
VM architecture x64
Agent status Ready
Agent version 2.14.0.1
Hibernation Disabled
Networking
Public IP address 74.234.32.49 (Network interface sagarvm452_z1)
Public IP address (IPv6) -
Private IP address 10.0.0.4
Private IP address (IPv6) -
Virtual network/subnet sagarvm-vnet/default
DNS name Configure

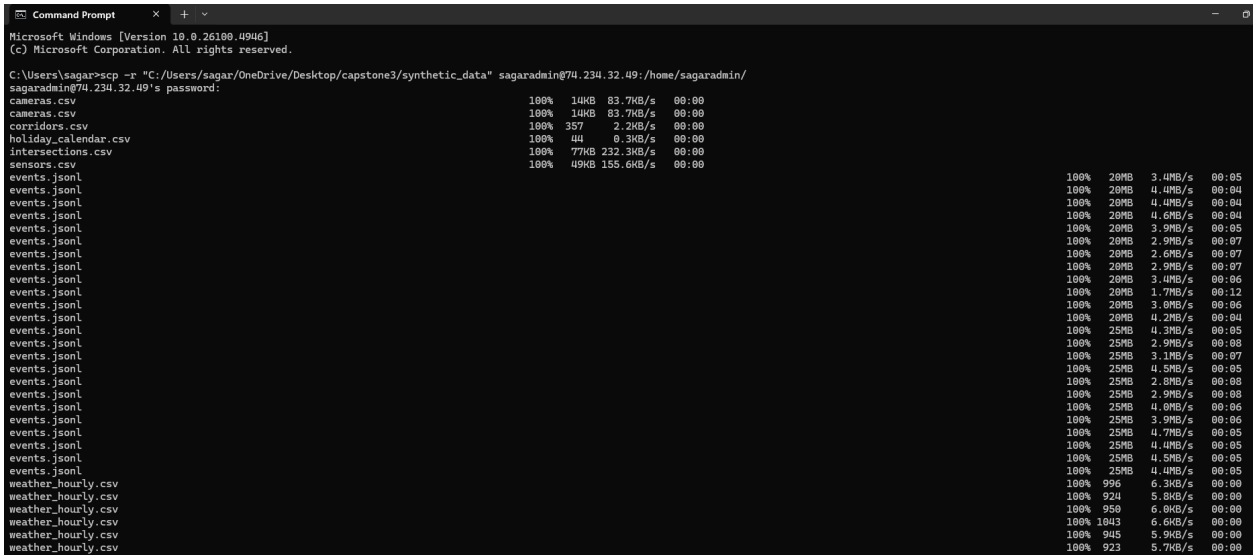
Connection to VM:



Libraries installation:



Copying files from local folder to VM:

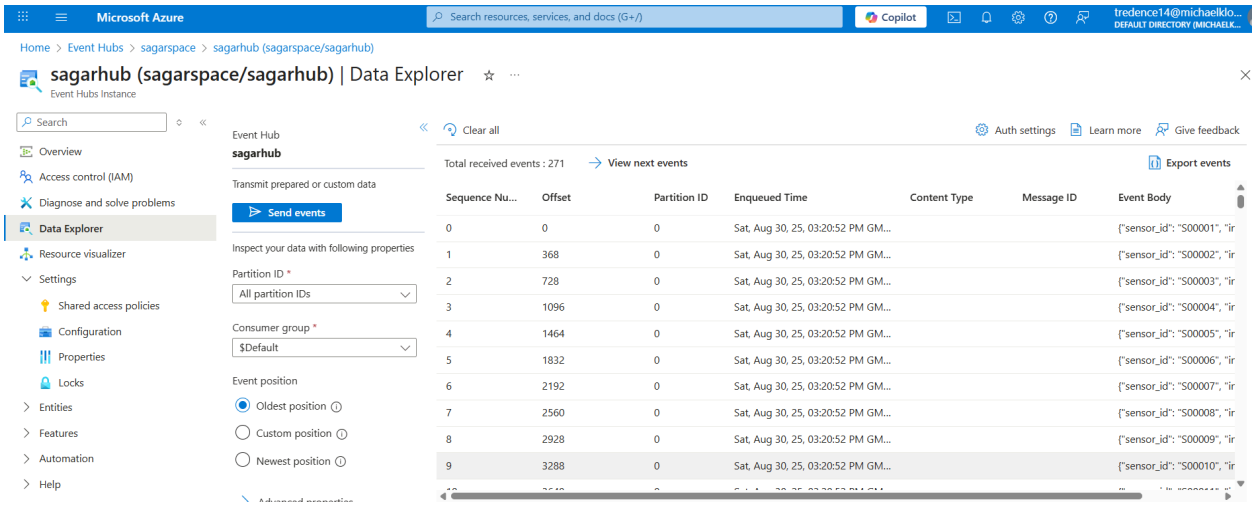


nano send.py

sending first events.jsonl(sensor_feed data) file for testing 👍

```
GNU nano 7.2
send.py
import os
import json
import time
from azure.eventhub import EventHubProducerClient, EventData

connection_str =
'Endpoint=sb://sagarspace.servicebus.windows.net/;SharedAccessKe
yName=sagarsas;SharedAccessKey=sB/y4LGadpDlaCDbN9QFiMTmnjy4qKELp
+AEhP2vyDM=;EntityPath=sagarhub'
eventhub_name = "sagarhub"
batch_size = 100

data_file_path =
"/home/sagaradmin/synthetic_data/stream/sensors/date=2025-08-29/
hour=00/events.jsonl"

try:
    if not connection_str:
        raise ValueError("EVENT_HUBS_CONNECTION_STRING
environment variable not set.")
```

```
    if not os.path.exists(data_file_path):
        raise FileNotFoundError(f"The file was not found:
{data_file_path}")

    producer =
EventHubProducerClient.from_connection_string(conn_str=connectio
n_str, eventhub_name=eventhub_name)

    events = []
    with producer:
        with open(data_file_path, 'r') as file:
            for line in file:
                line = line.strip()
                if line:
                    events.append(EventData(line))

                    if len(events) >= batch_size:
                        producer.send_batch(events)
                        print(f"Sent a batch of {len(events)}
events.")
                        events = []
                        time.sleep(1)

        if events:
            producer.send_batch(events)
            print(f"Sent final batch of {len(events)} events.")

    print("\nData streaming complete.")

except ValueError as e:
    print(f"Error: {e}")
except FileNotFoundError as e:
    print(f"Error: {e}")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```



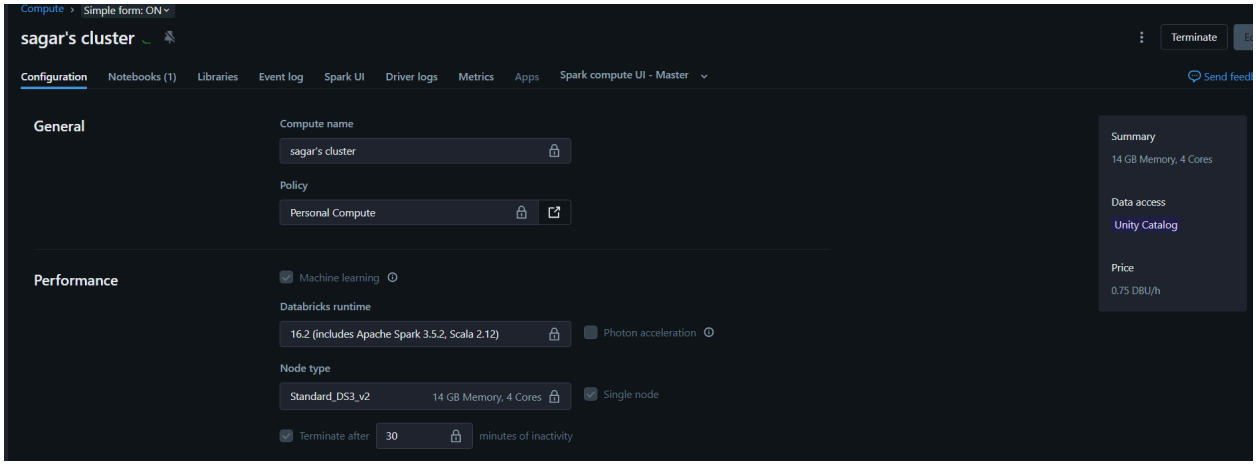
The error that we got while receiving the data from event-Hub into workspace:

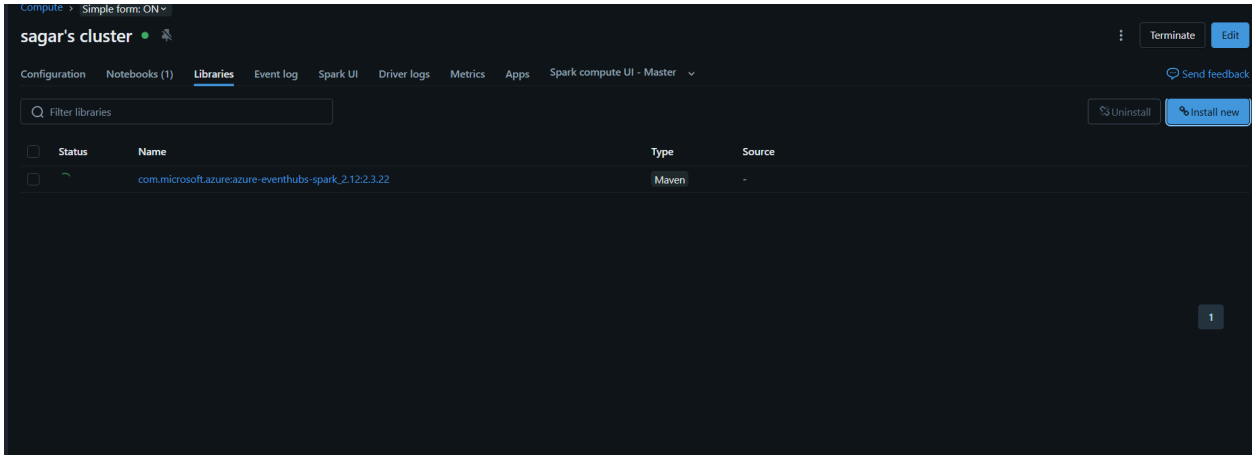
Issue:

The error occurs because **Event Hubs streaming source is not supported on a shared cluster**. Shared clusters restrict direct connections to external streaming services.

Solution:

Create and attach a **personal (single-user) cluster** with the required Spark Event Hubs library installed. Update your streaming job to run on this personal cluster instead of the shared one. This ensures proper permissions and allows the Event Hubs stream to be processed without restriction.





This below code establishes a **structured streaming pipeline** that ingests live traffic sensor data from **Azure Event Hubs** and stores it in **Delta tables** for analytics.

1 . **Spark Session Initialization**

A Spark session is created with the app name *EventHubStream* to handle streaming workloads.

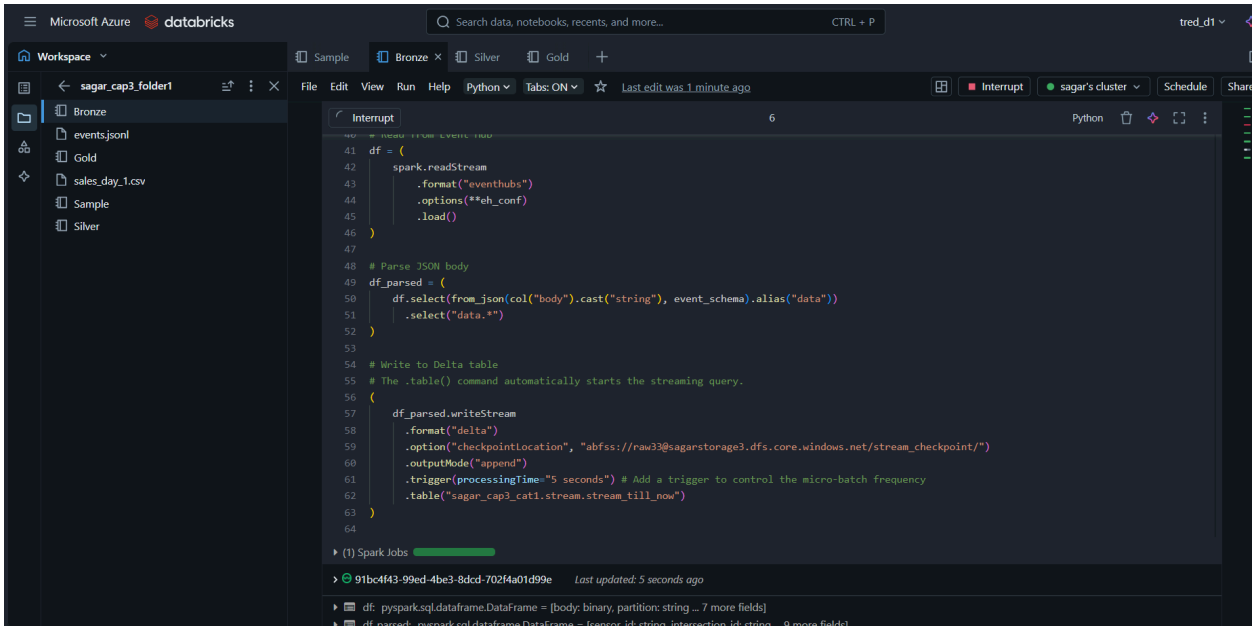
2. **Event Hubs Configuration**

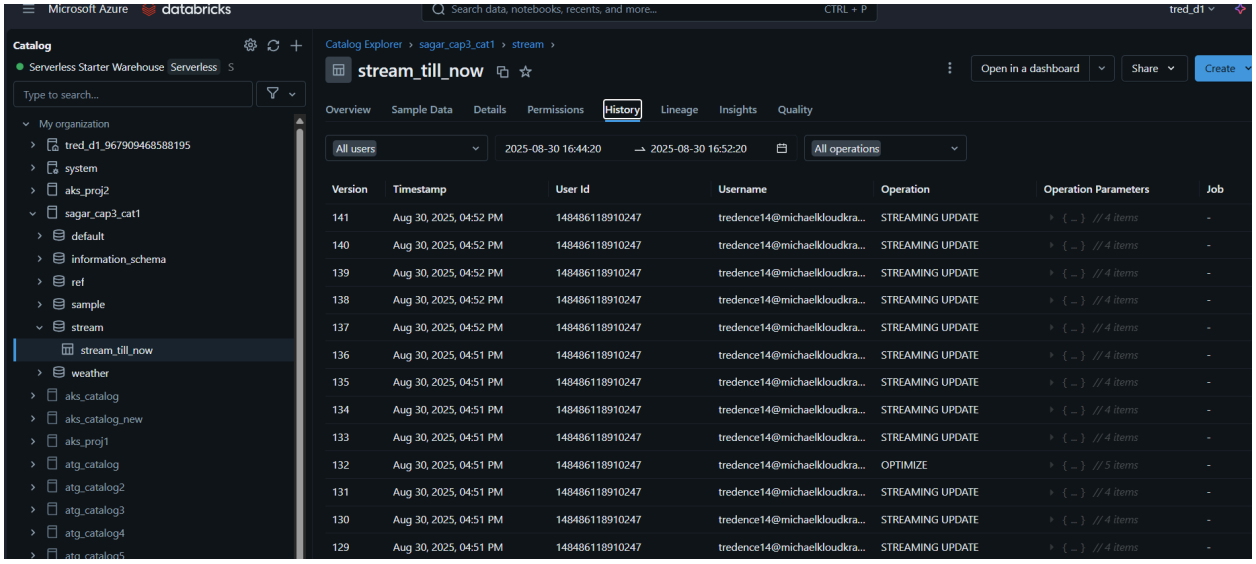
- The connection string to Azure Event Hubs is securely encrypted using EventHubsUtils.encrypt.
- This allows Spark to connect to the Event Hub (sagarhub) where live traffic events are being published.

3. **Schema Definition**
   - A structured schema (StructType) defines the expected JSON payload, including attributes like:
     - sensor_id, intersection_id, zone, corridor_id
     - Geolocation (lat, lon)
     - Timestamps (event_ts, ingest_ts)
     - Metrics (vehicle_count, avg_speed_kmh, lane_status, aqi)
4. This ensures that data is parsed correctly from raw JSON into structured columns.
5. **Reading from Event Hub**
   - The stream is read using .format("eventhubs").
   - Each event's **body** is extracted and converted into structured fields using from_json.
6. **Parsing the Data**
   - Raw JSON is parsed into the defined schema.

- The resulting DataFrame (df_parsed) contains clean, queryable traffic sensor data.

7. **Writing to Delta Table**
    - The parsed stream is written to a **Delta table** (sagar_cap3_cat1.streamin.stream_till_now) for reliable storage and querying.
    - Key configurations:
        - checkpointLocation: Stores streaming state for fault tolerance in ADLS Gen2.
        - outputMode("append"): Appends new data continuously.
        - .trigger(processingTime="5 seconds"): Ensures near real-time ingestion with 5-second micro-batches.

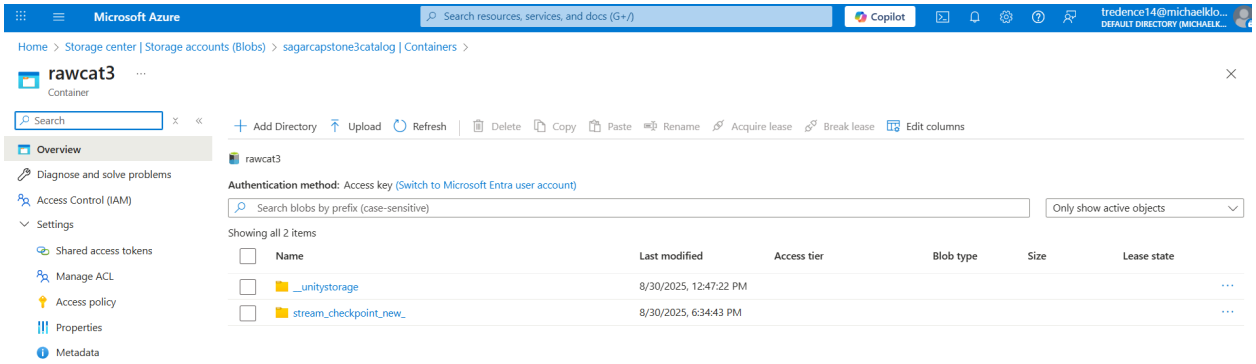




Previous 3 file hase been sent one by one from for testing and above is the result

Now we will change checkpoint location for fresh streaming of all data:

changed checkpoint location to my catalog storage:



Dropped table changed schema and created schema streamin.

Data is being sent from vm:

send.py on vm:

```
import os
import json
import time
from azure.eventhub import EventHubProducerClient, EventData

connection_str =
'Endpoint=sb://sagarspace.servicebus.windows.net/;SharedAccessKe
yName=sagarsas;SharedAccessKey=sB/y4LGadpDlaCDbN9QFiMTmnjy4qKELp
+AEhP2vyDM=;EntityPath=sagarhub'
eventhub_name = "sagarhub"
batch_size = 100

base_data_path =
"/home/sagaradmin/synthetic_data/stream/sensors/date=2025-08-29/
"

try:
    if not connection_str:
        raise ValueError("EVENT_HUBS_CONNECTION_STRING
environment variable not set.")
```

```
    producer =
EventHubProducerClient.from_connection_string(conn_str=connectio
n_str, eventhub_name=eventhub_name)

    for hour in range(24):
        hour_str = f"{hour:02d}"
        data_file_path = os.path.join(base_data_path,
f"hour={hour_str}", "events.jsonl")

        if not os.path.exists(data_file_path):
            print(f"Skipping hour={hour_str}, file not found:
{data_file_path}")
            continue

        print(f"Starting stream for file: {data_file_path}")
        events = []
        with producer:
            with open(data_file_path, 'r') as file:
                for line in file:
                    line = line.strip()
                    if line:
                        events.append(EventData(line))

                        if len(events) >= batch_size:
                            producer.send_batch(events)
                            print(f"Sent a batch of
{len(events)} events for hour={hour_str}.")
                            events = []
                            time.sleep(1)

        if events:
            producer.send_batch(events)
            print(f"Sent final batch of {len(events)} events for
hour={hour_str}.")

    print("\nAll data streaming complete.")

except ValueError as e:
    print(f"Error: {e}")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```

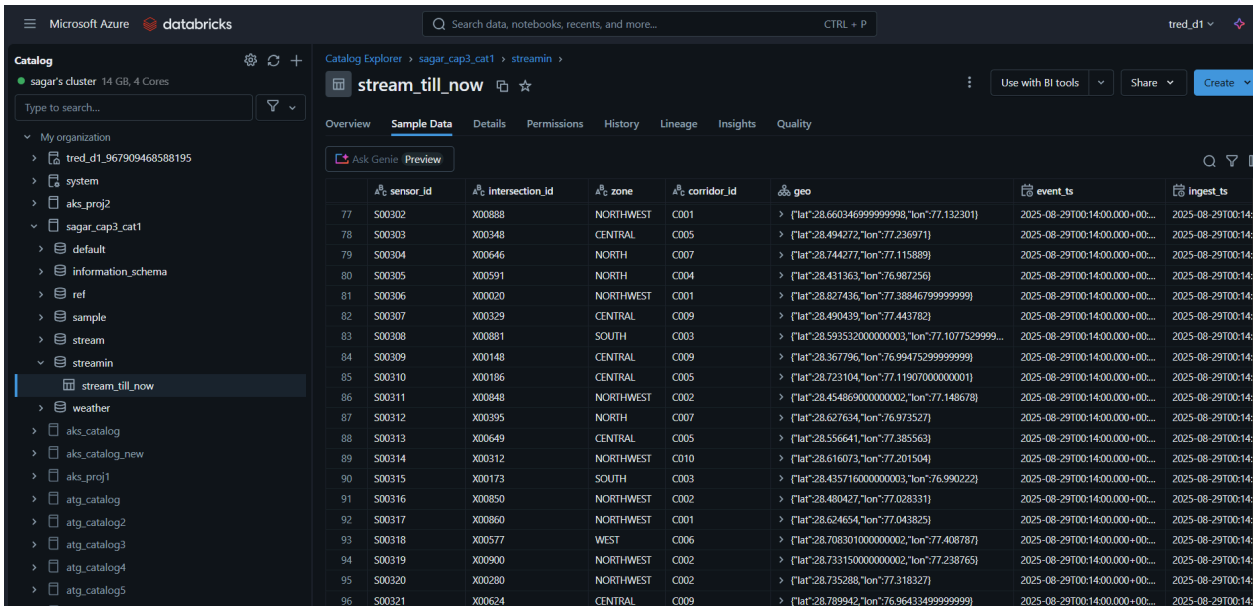
```
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent a batch of 100 events for hour=01.
Sent final batch of 91 events for hour=01.
Starting stream for file: /home/sagaradmin/synthetic_data/stream/sensors/date=2025-08-29/hour=02/events.jsonl
Sent a batch of 100 events for hour=02.
Sent a batch of 100 events for hour=02.
Sent a batch of 100 events for hour=02.
Sent a batch of 100 events for hour=02.
Sent a batch of 100 events for hour=02.
Sent a batch of 100 events for hour=02.
Sent a batch of 100 events for hour=02.
Sent a batch of 100 events for hour=02.
Sent a batch of 100 events for hour=02.
Sent a batch of 100 events for hour=02.
Sent a batch of 100 events for hour=02.
Sent a batch of 100 events for hour=02.
Sent a batch of 100 events for hour=02.
Sent a batch of 100 events for hour=02.
```

Data is being received into delta table:
Code for connecting to eventhub and receiving streaming data:

```
%sql
use catalog sagar_cap3_cat1;
create schema if not exists streamin;
use schema streamin;

import base64
from pyspark.sql import SparkSession
from pyspark.sql.functions import from_json, col
from pyspark.sql.types import StructType, StructField,
StringType, TimestampType, DoubleType, IntegerType


spark =
SparkSession.builder.appName("EventHubStream").getOrCreate()
```

```
event_hub_connection_string =
"Endpoint=sb://sagarspace.servicebus.windows.net/;SharedAccessKe
yName=sagarsas;SharedAccessKey=jsILtbhMkaE2rnhMICs1nm0FJhhf69wPy
+AEhFwzzOQ=;EntityPath=sagarhub"

eh_conf = {
    "eventhubs.connectionString":
spark._jvm.org.apache.spark.eventhubs.EventHubsUtils.encrypt(eve
nt_hub_connection_string)
}

# Define schema
event_schema = StructType([
    StructField("sensor_id", StringType(), True),
    StructField("intersection_id", StringType(), True),
    StructField("zone", StringType(), True),
    StructField("corridor_id", StringType(), True),
    StructField("geo", StructType([
        StructField("lat", DoubleType(), True),
        StructField("lon", DoubleType(), True)
    ]), True),
    StructField("event_ts", TimestampType(), True),
    StructField("ingest_ts", TimestampType(), True),
    StructField("vehicle_count", IntegerType(), True),
    StructField("avg_speed_kmh", DoubleType(), True),
    StructField("lane_status", StringType(), True),
    StructField("aqi", IntegerType(), True)
])

# Read from Event Hub
```

```
df = (
    spark.readStream
        .format("eventhubs")
        .options(**eh_conf)
        .load()
)


df_parsed = (
    df.select(from_json(col("body").cast("string"),
event_schema).alias("data"))
      .select("data.*")
)


(
    df_parsed.writeStream
      .format("delta")
      .option("checkpointLocation",
"abfss://rawcat3@sagarcapstone3catalog.dfs.core.windows.net/stre
am_checkpoint_new_/")
      .outputMode("append")
      .trigger(processingTime="5 seconds")
      .table("sagar_cap3_cat1.streamin.stream_till_now")
)
```



see this for streaming video:
https://youtu.be/WNgx2Cq5nRM?si=CK4fn5kPui1Ra3GO

After ingesting all the files from eventhub:

silver

```
%sql
use catalog sagar_cap3_cat1;
use schema silver
Data cleaning for cameras table
from pyspark.sql.functions import col, round, isnull, when

cameras_df = spark.table("sagar_cap3_cat1.ref.cameras")

cameras_df = cameras_df.select(
    col("camera_id").cast("string"),
    col("intersection_id").cast("string"),
    col("corridor_id").cast("string"),
    col("zone").cast("string"),
    col("fault_codes").cast("string"),
    round(col("latitude"), 2).alias("latitude"),
    round(col("longitude"), 2).alias("longitude"),
    round(col("uptime_pct"), 2).alias("uptime_pct"),
    col("maintenance_logs")
)

duplicates =
cameras_df.groupBy(cameras_df.columns).count().filter(col("count
") > 1)
null_check = cameras_df.filter(isnull(col("camera_id")) |
isnull(col("intersection_id")) |
                               isnull(col("corridor_id")) |
isnull(col("zone")) |
                               isnull(col("fault_codes")) |
isnull(col("latitude")) |
```

```
                                    isnull(col("longitude")) |
isnull(col("uptime_pct")) |
                                    isnull(col("maintenance_logs")))

cameras_df = (cameras_df.withColumn("uptime_pct",
when((col("uptime_pct") >= 0) & (col("uptime_pct") <= 100),
col("uptime_pct")).otherwise(None))
    .withColumn("latitude", when((col("latitude") >= -90) &
(col("latitude") <= 90), col("latitude")).otherwise(None))
    .withColumn("longitude", when((col("longitude") >= -180) &
(col("longitude") <= 180), col("longitude")).otherwise(None)))

cameras_df.write.mode('overwrite').saveAsTable('sagar_cap3_cat1.
silver.cameras')

Data cleaning for corridors table
from pyspark.sql.functions import col, round

corridors_df = spark.table("sagar_cap3_cat1.ref.corridors")

corridors_df = corridors_df.select(
    col("corridor_id").cast("string"),
    col("corridor_name").cast("string"),
    col("zone").cast("string"),
    round(col("length_km"), 2).alias("length_km")
)
corridors_df.write.mode('overwrite').saveAsTable('sagar_cap3_cat
1.silver.corridors')
display(corridors_df)

Data cleaning for holiday_calendar table
from pyspark.sql.functions import col, round
```

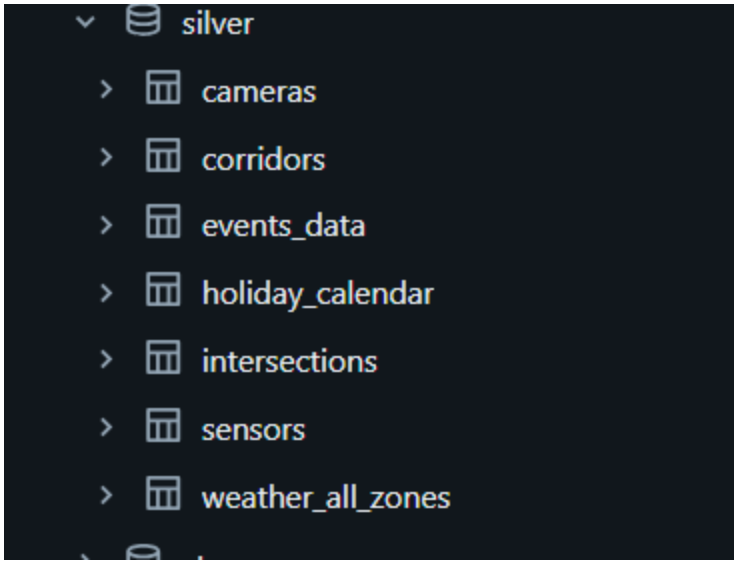
```
holiday_calendar =
spark.table("sagar_cap3_cat1.ref.holiday_calendar")
holiday_calendar=(holiday_calendar.withColumn('date',col('date')
.cast('date'))

.withColumn('holiday_name',col('holiday_name').cast('string')))
holiday_calendar.write.mode('overwrite').saveAsTable('sagar_cap3
_cat1.silver.holiday_calendar')
display(holiday_calendar)
Data cleaning for intersections table
from pyspark.sql.functions import col, round

intersections = spark.table("sagar_cap3_cat1.ref.intersections")

intersections = intersections.select(
    col("intersection_id").cast("string"),
    col("name").cast("string"),
    col("corridor_id").cast("string"),
    col("corridor_name").cast("string"),
    col("zone").cast("string"),
    round(col("latitude"), 2).alias("latitude"),
    round(col("longitude"), 2).alias("longitude"),
    col("school_distance_m").cast("int"),
    col("hospital_distance_m").cast("int"),
    col("near_school").cast("boolean"),
    col("near_hospital").cast("boolean"),
    col("is_top50").cast("boolean")
)
intersections.write.mode('overwrite').saveAsTable('sagar_cap3_ca
t1.silver.intersections')
display(intersections)
```

```
Data cleaning for sensors table
from pyspark.sql.functions import col, round, to_date

sensors = spark.table("sagar_cap3_cat1.ref.sensors")

sensors = sensors.select(
    col("sensor_id").cast("string"),
    col("intersection_id").cast("string"),
    col("corridor_id").cast("string"),
    col("zone").cast("string"),
    col("direction").cast("string"),
    col("model").cast("string"),
    col("firmware").cast("string"),
    col("lane_count").cast("int"),
    round(col("latitude"), 2).alias("latitude"),
    round(col("longitude"), 2).alias("longitude"),
    to_date(col("install_date"),
"yyyy-MM-dd").alias("install_date")
)
sensors.write.mode('overwrite').saveAsTable('sagar_cap3_cat1.sil
ver.sensors')
display(sensors)
Data cleaning for stream_till_now_table
from pyspark.sql.functions import col, round, to_timestamp,
mean, struct, when

stream_till_now =
spark.table("sagar_cap3_cat1.streamin.stream_till_now")

aqi_mean = stream_till_now.select(mean("aqi")).collect()[0][0]
occupancy_mean =
stream_till_now.select(mean("occupancy")).collect()[0][0]
```

```
battery_level_mean =
stream_till_now.select(mean("battery_level")).collect()[0][0]


processed_df = stream_till_now.select(
    col("sensor_id").cast("string"),
    col("intersection_id").cast("string"),
    col("zone").cast("string"),
    col("corridor_id").cast("string"),
    when(col("lane_status").isNull(),
"unknown").otherwise(col("lane_status")).cast("string").alias("l
ane_status"),
    when(col("speed_unit").isNull(),
"km/h").otherwise(col("speed_unit")).cast("string").alias("speed
_unit"),
    col("vehicle_count").cast("int"),
    when(col("aqi").isNull(),
aqi_mean).otherwise(col("aqi")).cast("int").alias("aqi"),
    round(when(col("occupancy").isNull(),
occupancy_mean).otherwise(col("occupancy")),
2).alias("occupancy"),
    round(when(col("battery_level").isNull(),
battery_level_mean).otherwise(col("battery_level")),
2).alias("battery_level"),
    round(col("avg_speed_kmh"), 2).alias("avg_speed_kmh"),
    to_timestamp(col("event_ts")).alias("event_ts"),
    to_timestamp(col("ingest_ts")).alias("ingest_ts"),
    struct(
        round(col("geo").getItem("lat"), 2).alias("lat"),
        round(col("geo").getItem("lon"), 2).alias("lon")
    ).alias("geo")
)
```

```
processed_df = processed_df.dropDuplicates()

display(processed_df)

processed_df.write.mode("overwrite").saveAsTable("sagar_cap3_cat
1.silver.events_data")
```

```
Data cleaning of weather_all_zones table:
from pyspark.sql.functions import col, to_timestamp

weather_all =
spark.table("sagar_cap3_cat1.weather.weather_all_zones")

weather_all = weather_all.select(
    col("timestamp").cast("timestamp").alias("timestamp"),
    col("zone").cast("string").alias("zone"),
    col("precip_mm").cast("float").alias("precip_mm"),
    col("temperature_c").cast("float").alias("temperature_c")
)

weather_all.write.mode("overwrite").saveAsTable("sagar_cap3_cat1
.silver.weather_all_zones")
display(weather_all)
A glimpse of silver tables created:
```



GOLD

```
# Table : gold_daily_weather_traffic_correlation
# Description: Aggregates daily weather and traffic data by zone

from pyspark.sql.functions import avg, sum, to_date, col, round

stream_df = spark.table("sagar_cap3_cat1.silver.events_data")
weather_df =
spark.table("sagar_cap3_cat1.silver.weather_all_zones")

daily_correlation_agg = (
    stream_df.join(weather_df, on="zone", how="left")
    .where(to_date(col("event_ts")) ==
to_date(col("timestamp")))
    .groupBy(
        to_date(col("event_ts")).alias("report_date"),
        col("zone")
    )
    .agg(
        round(avg("avg_speed_kmh"), 2).alias("avg_speed_kph"),
```

```
        sum("vehicle_count").alias("daily_vehicle_count"),
        round(avg("aqi"), 2).alias("avg_aqi"),
        round(avg("precip_mm"), 2).alias("avg_precipitation"),
        round(avg("temperature_c"),
2).alias("avg_temperature_c")
    )
    .orderBy("report_date")
)
daily_correlation_agg.write.mode('overwrite').saveAsTable('sagar
_cap3_cat1.gold.gold_daily_weather_traffic_correlation')
display(daily_correlation_agg)


## intersection_aqi_summary
from pyspark.sql.functions import avg, sum, to_date, col

intersection_aqi_summary = (
    spark.table("sagar_cap3_cat1.silver.events_data")
    .groupBy(
        to_date("event_ts").alias("report_date"),
        col("intersection_id")
    )
    .agg(
        round(avg("aqi"), 2).alias("daily_avg_aqi"),
        sum("vehicle_count").alias("daily_vehicle_count"),
        round(avg("avg_speed_kmh"),
2).alias("daily_avg_speed_kph")
    )
    .orderBy("report_date", "intersection_id")
)
```

```
intersection_aqi_summary.write.mode("overwrite").saveAsTable("sa
gar_cap3_cat1.gold.intersection_aqi_summary")
display(intersection_aqi_summary)


# Table name: gold_intraday_traffic_summary

from pyspark.sql.functions import sum, avg, window, col

intraday_traffic_summary = (
    stream_df.groupBy(
        window("event_ts", "15 minutes").alias("window"),
        "intersection_id",
        "corridor_id",
        "zone"
    )
    .agg(
        sum("vehicle_count").alias("15_min_vehicle_count"),
        avg("avg_speed_kmh").alias("15_min_avg_speed_kph"),
        avg("aqi").alias("15_min_avg_aqi")
    )
    .orderBy("window", "intersection_id")
)
intraday_traffic_summary.write.mode("overwrite").saveAsTable("sa
gar_cap3_cat1.gold.gold_intraday_traffic_summary")
display(intraday_traffic_summary)




### corridor_traffic_by_hour
```

```
from pyspark.sql.functions import sum, avg, hour, col

hourly_corridor_traffic = (
    spark.table("sagar_cap3_cat1.silver.events_data")
    .groupBy(
        col("corridor_id"),
        hour("event_ts").alias("hour_of_day")
    )
    .agg(
        sum("vehicle_count").alias("total_hourly_vehicles"),

round(avg("avg_speed_kmh"),2).alias("avg_hourly_speed_kph")  #
m2
    )
    .orderBy("corridor_id", "hour_of_day")
)

hourly_corridor_traffic.write.mode("overwrite").saveAsTable("sag
ar_cap3_cat1.gold.corridor_traffic_by_hour")
display(hourly_corridor_traffic)


# Table: critical_location_impact
# Description: Analyzes traffic impact near critical locations
like schools and hospitals

from pyspark.sql.functions import sum, avg, hour, col, when,
to_date

stream_df = spark.table("sagar_cap3_cat1.silver.events_data")
intersections_df =
spark.table("sagar_cap3_cat1.silver.intersections")
```

```
critical_location_impact = (
    stream_df.join(intersections_df, on="intersection_id")
    .where((col("near_school") == True) | (col("near_hospital")
== True))
    .groupBy(
        col("intersection_id"),
        to_date("event_ts").alias("report_date"),
        hour("event_ts").alias("hour_of_day"),
        when(col("near_school") == True,
"School").otherwise(None).alias("near_school"),
        when(col("near_hospital") == True,
"Hospital").otherwise(None).alias("near_hospital")
    )
    .agg(
        avg("avg_speed_kmh").alias("avg_speed_kph"),
        sum("vehicle_count").alias("total_vehicles"),
        avg("aqi").alias("avg_aqi_near_location")
    )
    .orderBy("report_date", "hour_of_day")
)

critical_location_impact.write.mode("overwrite").saveAsTable("sa
gar_cap3_cat1.gold.critical_location_impact")
display(critical_location_impact)


## Table: critical_locations_agg
## Hourly Congestion Near Critical Locations (Schools/Hospitals)
from pyspark.sql.functions import avg, sum, hour, when, col

stream_df = spark.table("sagar_cap3_cat1.silver.events_data")
```

```
intersections_df =
spark.table("sagar_cap3_cat1.silver.intersections")

critical_locations_agg = (
    stream_df.join(intersections_df, on="intersection_id")
    .where((col("near_school") == True) | (col("near_hospital")
== True))
    .groupBy(
        col("intersection_id"),
        hour("event_ts").alias("hour_of_day"),
        when(col("near_school") == True,
"School").otherwise(None).alias("near_school"),
        when(col("near_hospital") == True,
"Hospital").otherwise(None).alias("near_hospital")
    )
    .agg(
        avg("avg_speed_kmh").alias("avg_speed_kph"),
        sum("vehicle_count").alias("total_vehicles"),
        avg("aqi").alias("avg_aqi_near_location")
    )
    .orderBy("hour_of_day", "avg_aqi_near_location",
ascending=False)
)
critical_locations_agg.write.mode('overwrite').saveAsTable('saga
r_cap3_cat1.gold.critical_locations_agg')
display(critical_locations_agg)

# Table: gold_holiday_traffic_impact
# Description: Analyzes traffic impact on holidays vs. normal
days
```

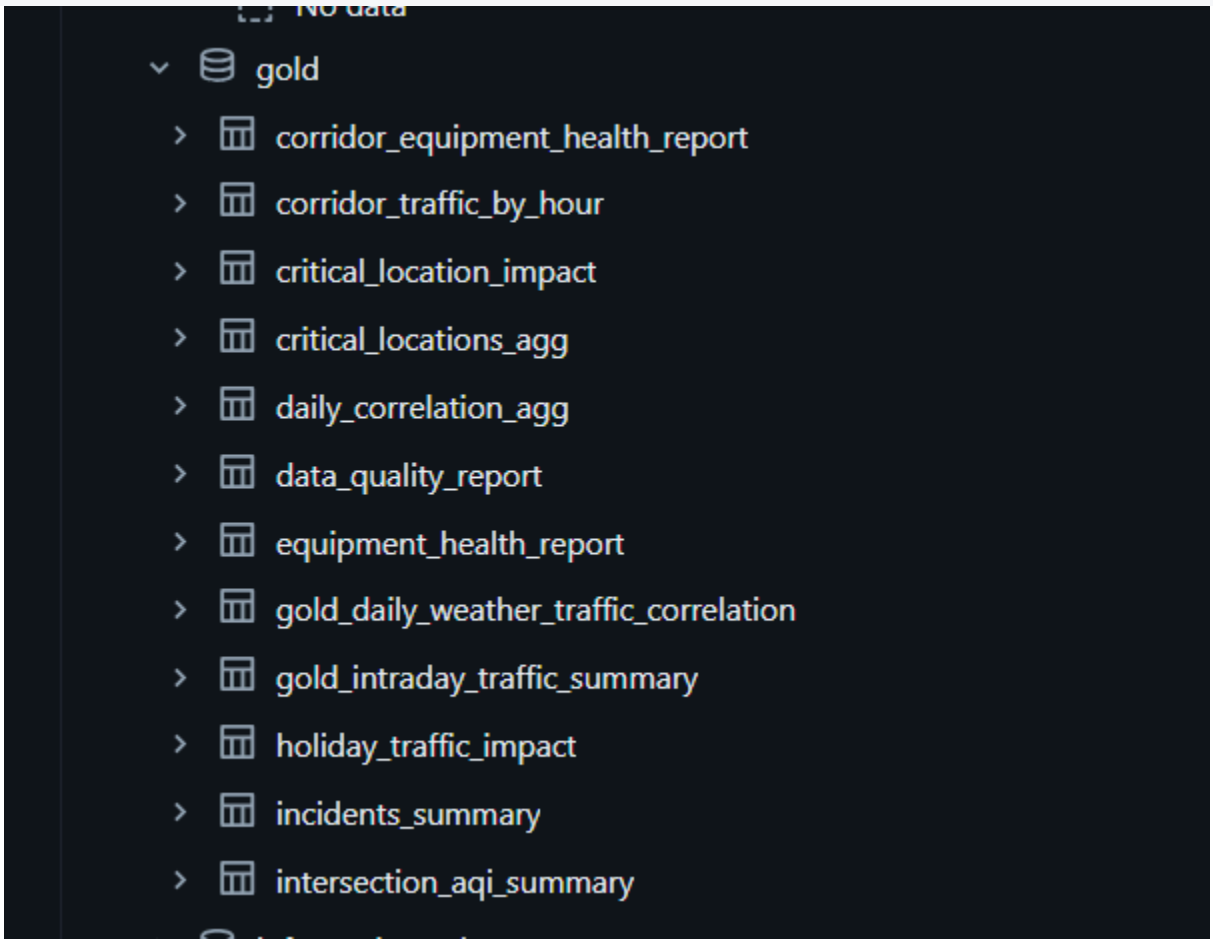
```
from pyspark.sql.functions import avg, sum, to_date, when, col,
hour

stream_df = spark.table("sagar_cap3_cat1.silver.events_data")
holidays_df =
spark.table("sagar_cap3_cat1.silver.holiday_calendar")

holiday_comparison_agg = (
    stream_df.join(holidays_df, to_date(col("event_ts")) ==
col("date"), how="left")
    .groupBy(
        when(col("holiday_name").isNotNull(),
"Holiday").otherwise("Normal Day").alias("day_type"),
        hour("event_ts").alias("hour_of_day")
    )
    .agg(
        round(avg("avg_speed_kmh"),2).alias("avg_speed_kph"),
        sum("vehicle_count").alias("total_vehicles")
    )
    .orderBy("day_type", "hour_of_day")
)

holiday_comparison_agg.write.mode("overwrite").saveAsTable("saga
r_cap3_cat1.gold.holiday_traffic_impact")
display(holiday_comparison_agg)



# Table: incidents_summary
# Description: Summarizes traffic incidents based on speed and
sensor status
```

```
from pyspark.sql.functions import when, lit, col, lag,
unix_timestamp
from pyspark.sql.window import Window

stream_df = spark.table("sagar_cap3_cat1.silver.events_data")

window_spec =
Window.partitionBy("sensor_id").orderBy("event_ts")
incidents = (
    stream_df
    .withColumn("prev_speed", lag("avg_speed_kmh",
1).over(window_spec))
    .withColumn(
        "time_diff_sec",
        unix_timestamp("event_ts") -
unix_timestamp(lag("event_ts", 1).over(window_spec))
    )
    .withColumn("speed_drop", col("prev_speed") -
col("avg_speed_kmh"))
    .withColumn(
        "incident_type",
        when((col("time_diff_sec") < 60) & (col("speed_drop") >
40), "Abrupt Speed Collapse")
        .when(col("lane_status") == "offline", "Sensor Offline")
        .otherwise(lit("None"))
    )
    .filter(col("incident_type") != "None")
    .select(
        col("event_ts").alias("timestamp"),
        col("incident_type"),
        col("aqi"),
        col("avg_speed_kmh"),
```

```
        col("geo"),
        lit("1.0").alias("severity_score")
    )
)

incidents.write.mode("append").saveAsTable("sagar_cap3_cat1.gold
.incidents_summary")
display(incidents)

## Table:corridor_equipment_health_report
from pyspark.sql.functions import avg, count, countDistinct, col

cameras_df = spark.table("sagar_cap3_cat1.silver.cameras")
sensors_df = spark.table("sagar_cap3_cat1.silver.sensors")

camera_health = (
    cameras_df.groupBy("corridor_id")
    .agg(

round(avg("uptime_pct"),2).alias("avg_camera_uptime_pct"),
        countDistinct("fault_codes").alias("num_camera_faults")
    )
)

sensor_health = (
    sensors_df.groupBy("corridor_id")
    .agg(
        count("sensor_id").alias("total_sensors"),
        countDistinct("firmware").alias("num_firmware_versions")
    )
)
```

```
equipment_health_report = (
    camera_health.join(sensor_health, on="corridor_id",
how="outer")
)

equipment_health_report.write.mode("overwrite").saveAsTable("sag
ar_cap3_cat1.gold.corridor_equipment_health_report")
display(equipment_health_report)



## data_quality_report
from pyspark.sql.functions import count, countDistinct, to_date,
col, lit

daily_data_quality = (
    spark.table("sagar_cap3_cat1.silver.events_data")
    .groupBy(to_date("event_ts").alias("report_date"))
    .agg(
        count("*").alias("total_events"),
        countDistinct("sensor_id").alias("unique_sensors"),
        sum(when(col("vehicle_count").isNull(),
1).otherwise(0)).alias("null_vehicle_counts"),
        sum(when(col("avg_speed_kmh").isNull(),
1).otherwise(0)).alias("null_speed_counts"),
        sum(when(col("aqi").isNull(),
1).otherwise(0)).alias("null_aqi_counts")
    )
    .withColumn("data_completeness_pct", (col("total_events") -
col("null_vehicle_counts") - col("null_speed_counts") -
col("null_aqi_counts")) / col("total_events") * 100)
    .orderBy("report_date")
)
```

```
daily_data_quality.write.mode("overwrite").saveAsTable("sagar_cap3_cat1.gold.data_quality_report")
display(daily_data_quality)
```

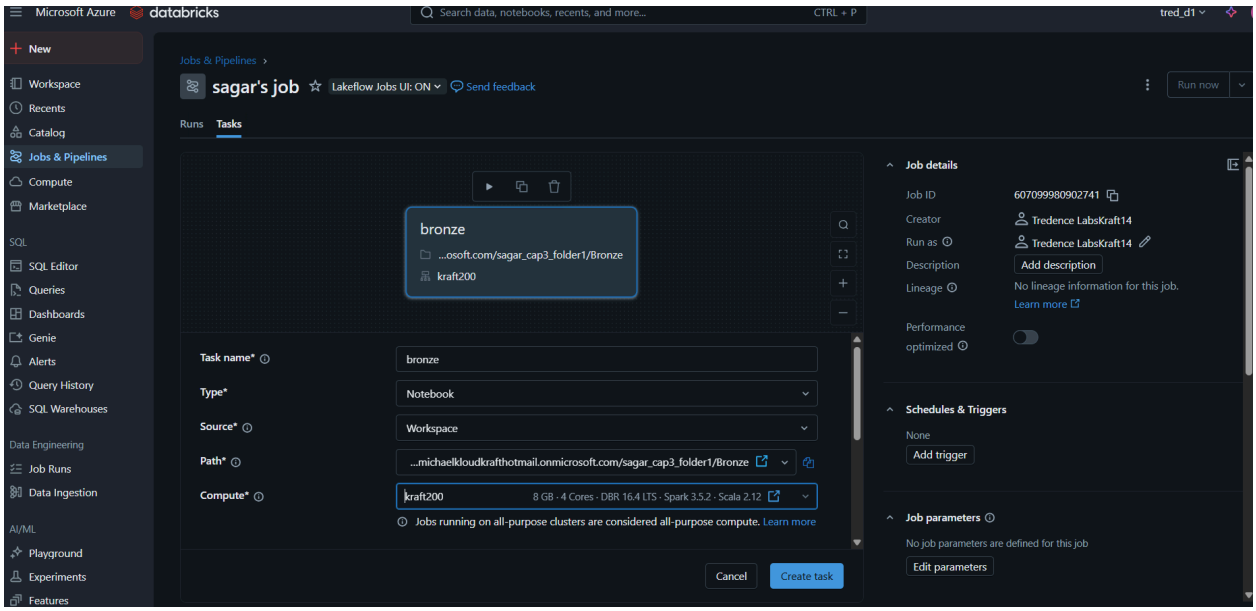


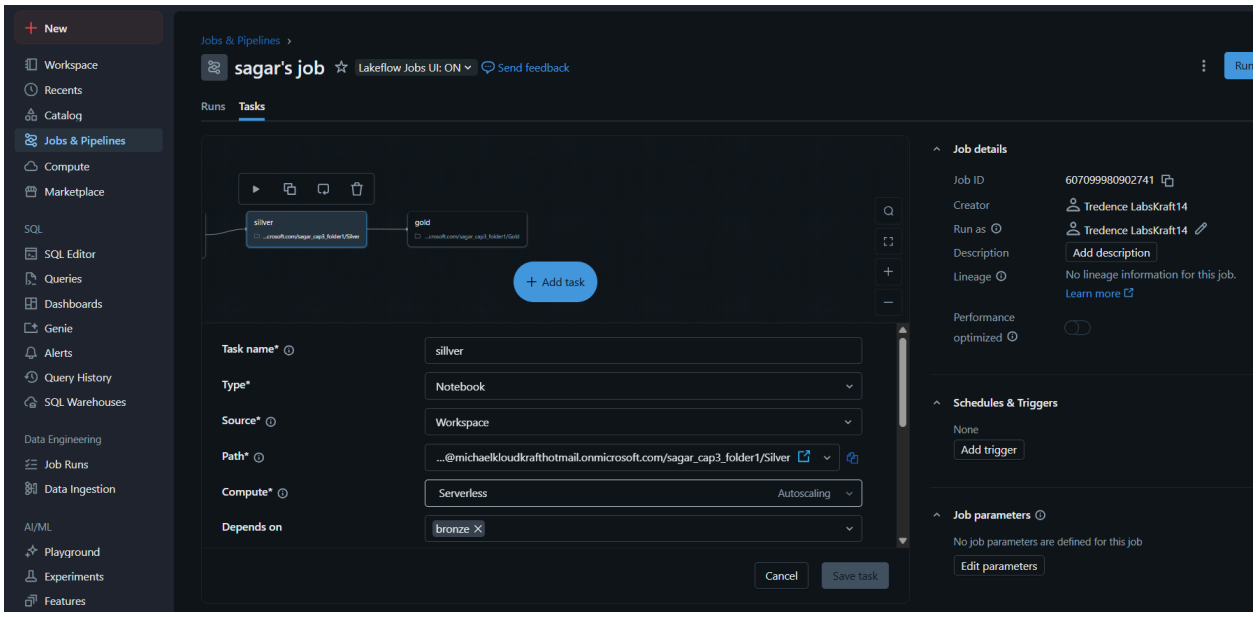
⚡ Job Description

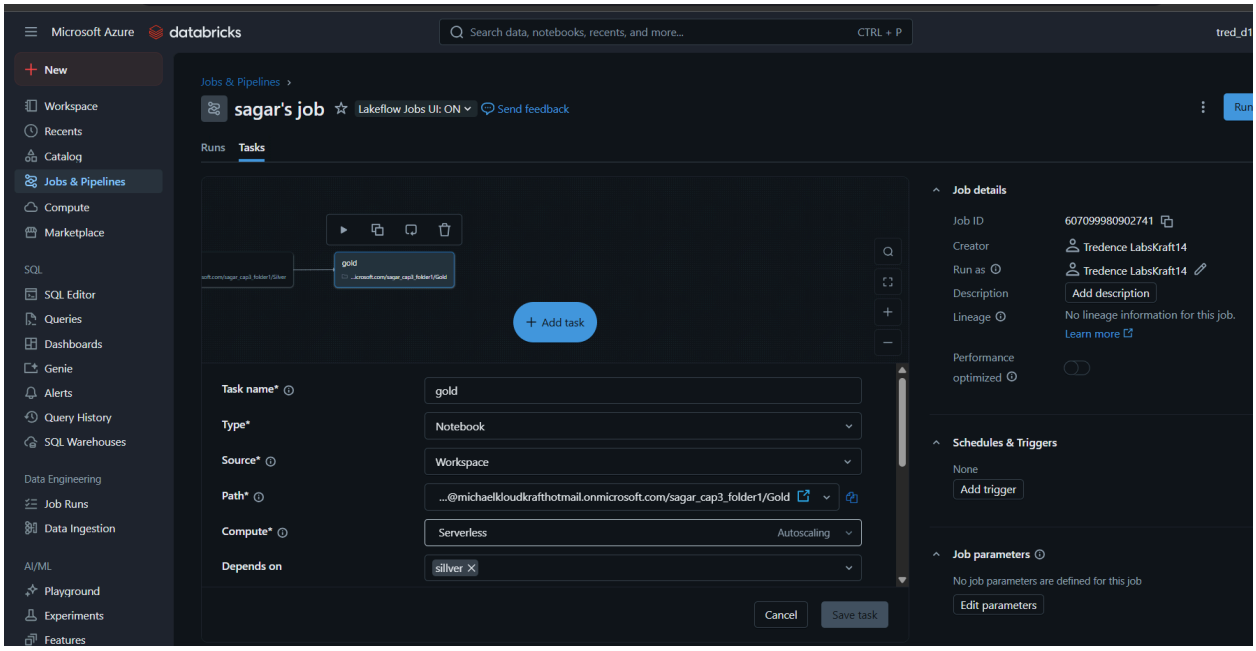
This Databricks workflow orchestrates a **Bronze → Silver → Gold** pipeline that ingests raw traffic data, applies transformations, and delivers curated analytics tables.

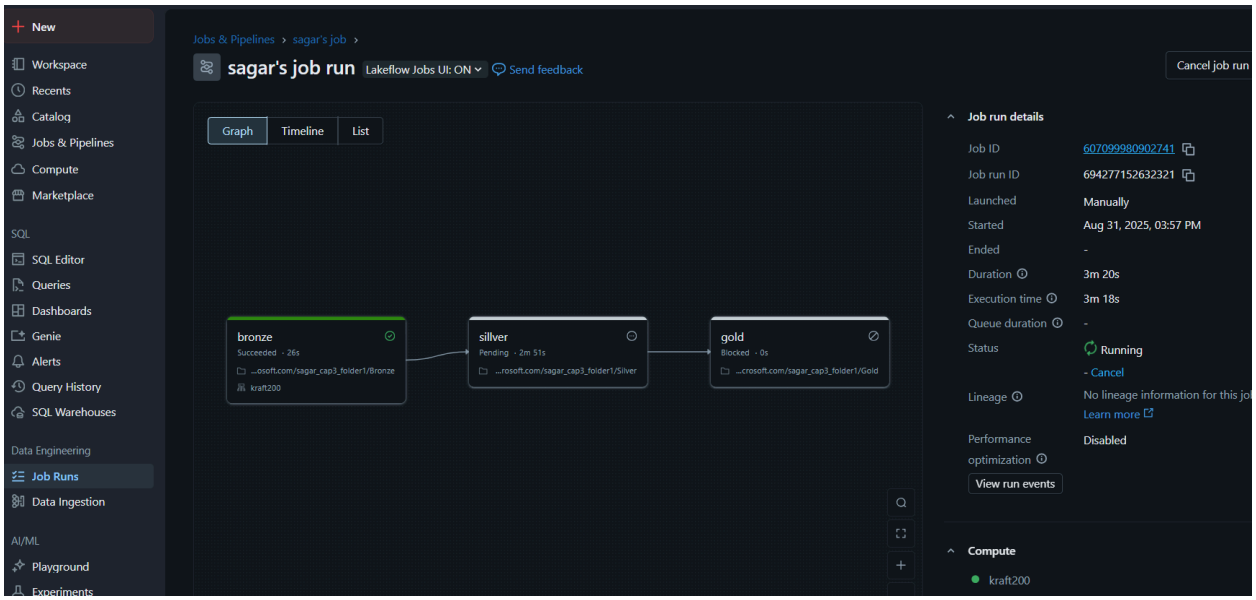
The job ensures seamless data flow with dependencies across stages, enabling reliable reporting, forecasting, and BI consumption.

It is monitored through job run details for execution time, status, and lineage.




New
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie
Alerts
Query History
SQL Warehouses
Data Engineering
Job Runs
Data Ingestion
AI/ML
Playground
Experiments
Features
Jobs & Pipelines
sagar's job
Lakeflow Jobs UI: ON
Send feedback
Runs
Tasks
sillver
gold
Add task
Task name*
sillver
Type*
Notebook
Source*
Workspace
Path*
...@michaelkloudkrafthotmail.onmicrosoft.com/sagar_cap3_folder1/Silver
Compute*
Serverless
Autoscaling
Depends on
bronze
Cancel
Save task
Job details
Job ID
607099980902741
Creator
Tredence LabsKraft14
Run as
Tredence LabsKraft14
Description
Add description
Lineage
No lineage information for this job.
Learn more
Performance optimized
Schedules & Triggers
None
Add trigger
Job parameters
No job parameters are defined for this job
Edit parameters


Microsoft Azure
databricks
Search data, notebooks, recents, and more...
CTRL + P
New
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie
Alerts
Query History
SQL Warehouses
Data Engineering
Job Runs
Data Ingestion
AI/ML
Playground
Experiments
Features
Jobs & Pipelines
sagar's job
Lakeflow Jobs UI: ON
Send feedback
Runs
Tasks
gold
Add task
Task name*
gold
Type*
Notebook
Source*
Workspace
Path*
...@michaelkloudkrafthotmail.onmicrosoft.com/sagar_cap3_folder1/Gold
Compute*
Serverless
Autoscaling
Depends on
sillver
Cancel
Save task
Job details
Job ID
607099980902741
Creator
Tredence LabsKraft14
Run as
Tredence LabsKraft14
Description
Add description
Lineage
No lineage information for this job.
Learn more
Performance optimized
Schedules & Triggers
None
Add trigger
Job parameters
No job parameters are defined for this job
Edit parameters


New
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie
Alerts
Query History
SQL Warehouses
Data Engineering
Job Runs
Data Ingestion
AI/ML
Playground
Experiments
Jobs & Pipelines
sagar's job
sagar's job run
Lakeflow Jobs UI: ON
Send feedback
Cancel job run
Graph
Timeline
List
bronze
Succeeded · 26s
kraft200
sillver
Pending · 2m 51s
gold
Blocked · 0s
Job run details
Job ID
607099980902741
Job run ID
694277152632321
Launched
Manually
Started
Aug 31, 2025, 03:57 PM
Ended
-
Duration
3m 20s
Execution time
3m 18s
Queue duration
-
Status
Running
Cancel
Lineage
No lineage information for this
Learn more
Performance optimization
Disabled
View run events
Compute
kraft200

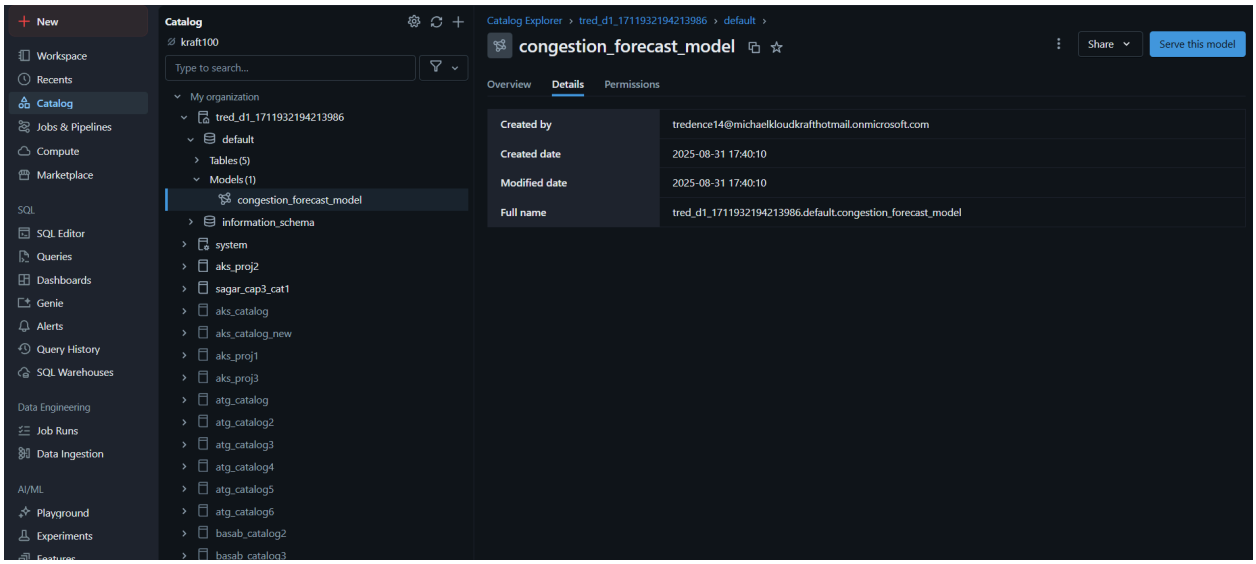
Congestion Model

1. **Catalog & Schema Setup**
    - Catalog: `sagar_cap3_cat1`
    - Schema: `models` created/used for storing ML models.
2. **Data Preparation**
    - From **Silver Layer**:
        - `intersections` table → filtered for **Top 50 intersections** (`is_top50=True`).
        - `events_data` table → joined with top intersections.
    - Converted Spark DataFrame to **Pandas** for ML model training.
    - Features: `vehicle_count`
    - Target: `avg_speed_kmh`
3. **Model Training**
    - Algorithm: **RandomForestRegressor**
    - Parameters: `n_estimators=100`, `max_depth=5`
    - Train/Test split: 80/20

4. **MLflow Integration**
    - Used **MLflow** to:
        - Log model artifacts, signature (input/output schema), and example input.
        - Register model under: `sagar_cap3_cat1.models.congestion_forecast`
5. **Model Registry Check**
    - Used `MlflowClient()` to list all model versions and their current stages.

```
%sql
use catalog sagar_cap3_cat1;
CREATE SCHEMA IF NOT EXISTS models;
use catalog sagar_cap3_cat1;
USE SCHEMA models;

from pyspark.sql.functions import col
top_intersections =
spark.table("sagar_cap3_cat1.silver.intersections").filter(col("
is_top50") == True)
df=spark.table('sagar_cap3_cat1.silver.events_data')
top_intersections_stream = df.join(top_intersections,
on="intersection_id", how="inner")
display(top_intersections_stream)
```

```
import mlflow
import mlflow.sklearn
from mlflow.models import infer_signature
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor

# Prepare data
pdf = top_intersections_stream.toPandas()
X = pdf[["vehicle_count"]]
y = pdf["avg_speed_kmh"]

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Train model
model = RandomForestRegressor(n_estimators=100, max_depth=5)
model.fit(X_train, y_train)

# Infer model signature (input/output schema)
signature = infer_signature(X_train, model.predict(X_train))

with mlflow.start_run():
    mlflow.sklearn.log_model(
        sk_model=model,
        artifact_path="model",

registered_model_name="sagar_cap3_cat1.models.congestion_forecas
t",
        signature=signature,
        input_example=X_train.iloc[:5]   # optional but
recommended
    )
```




```
#checking the version of my model:
from mlflow import MlflowClient
client = MlflowClient()

for mv in
client.search_model_versions("name='sagar_cap3_cat1.models.conge
stion_forecast'"):
    print(f"Version: {mv.version}, Stage: {mv.current_stage}")
```

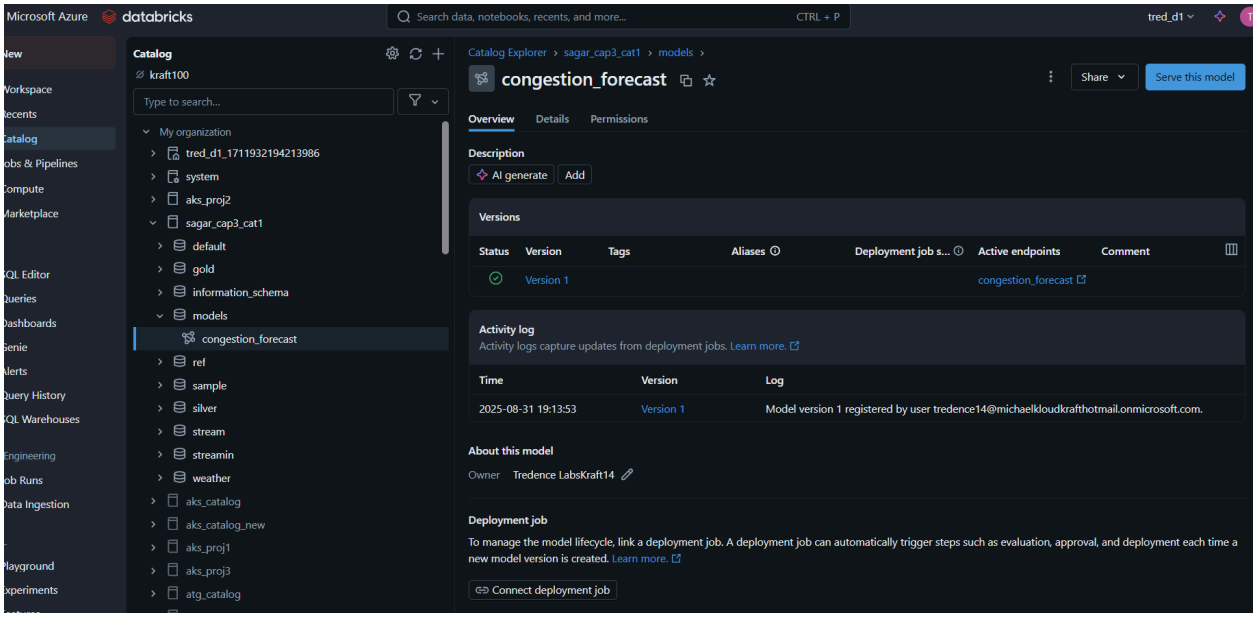
6

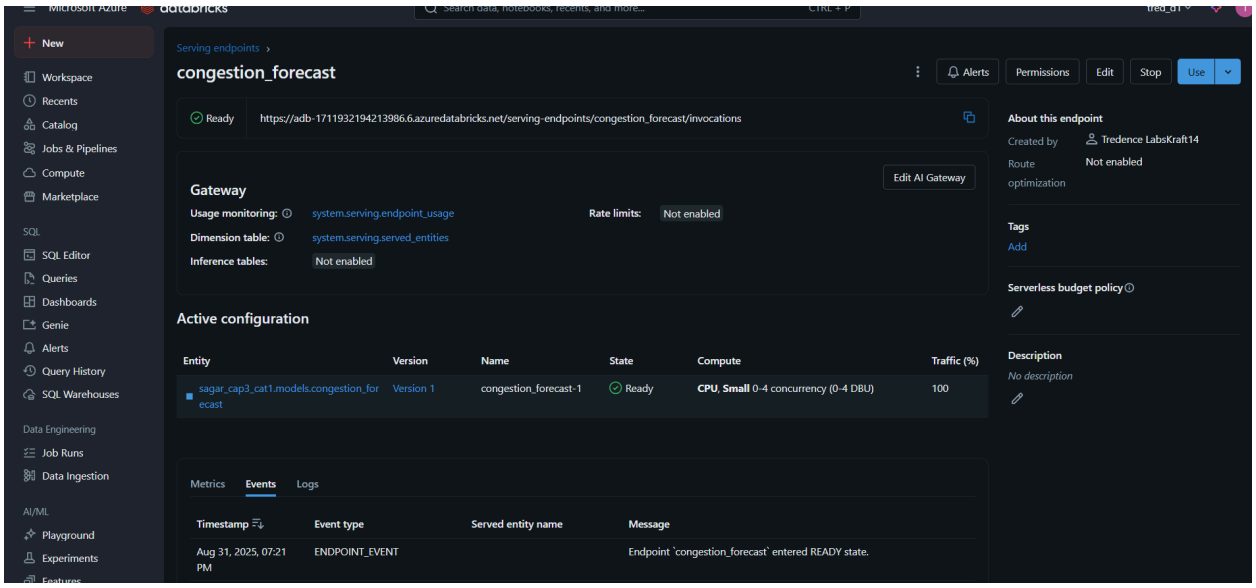
Python

```
import mlflow
import mlflow.pyfunc
import pandas as pd

# Sample data (replace this with your own test data)
data = {
    "vehicle_count": [100, 200, 300]
}
df = pd.DataFrame(data).astype({'vehicle_count': 'int32'})

# Model name
model_name = "congestion_forecast"

# Load latest version of the model
model = mlflow.pyfunc.load_model(f"models:/{model_name}/1")

# Run prediction
predictions = model.predict(df)

print("Predictions:")
print(predictions)
```

```
Predictions:
[55.71514768 55.71514768 55.71514768]
```

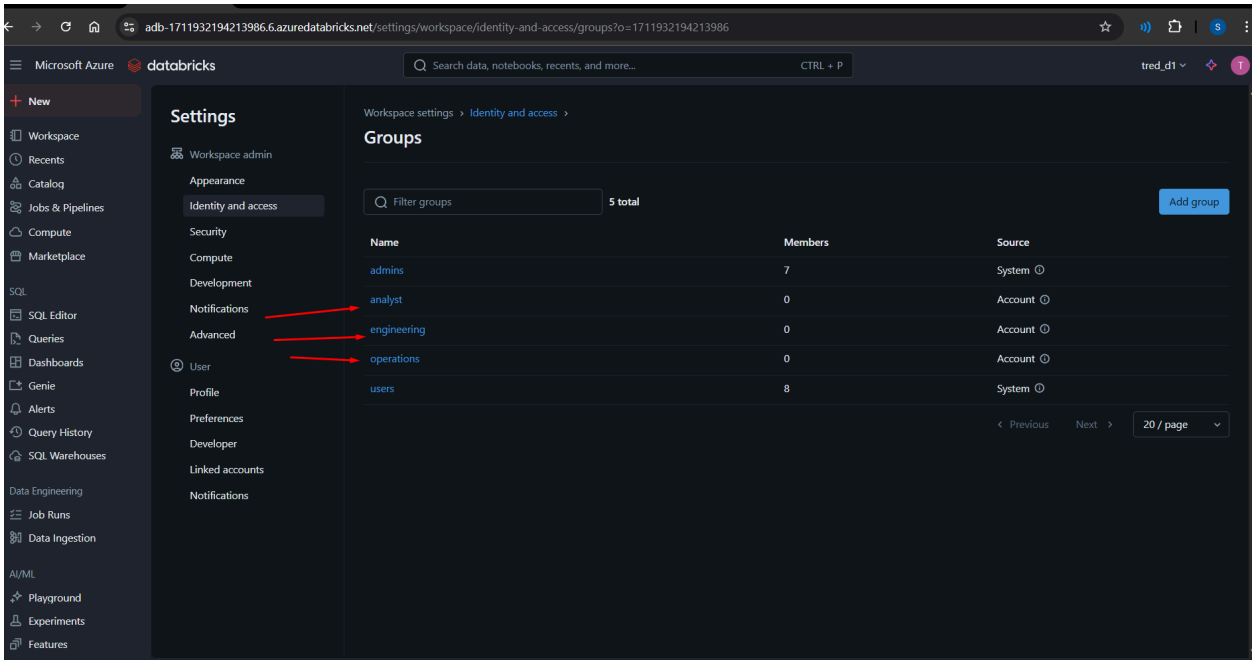
Serving this model:

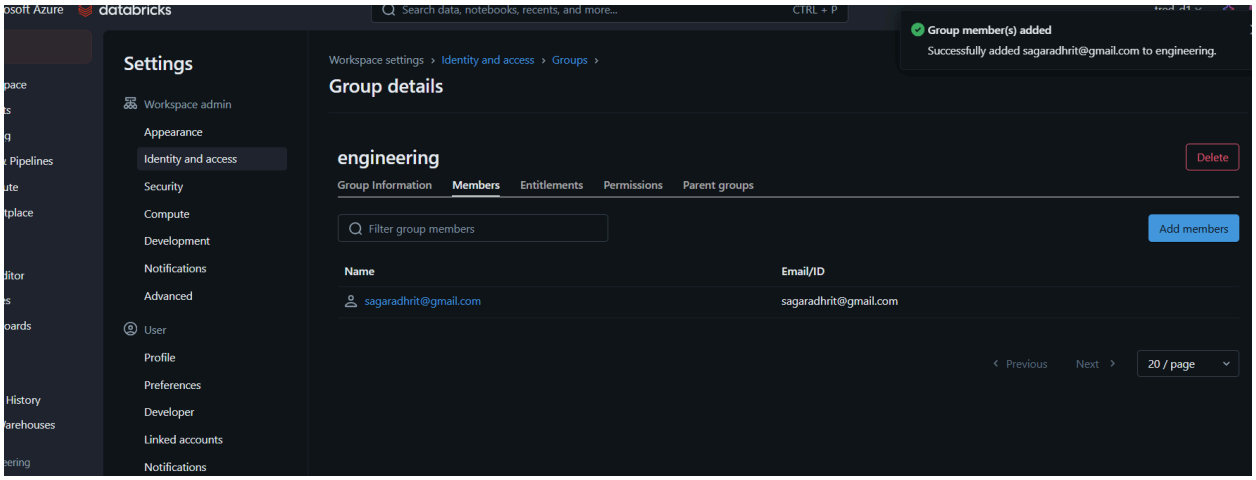


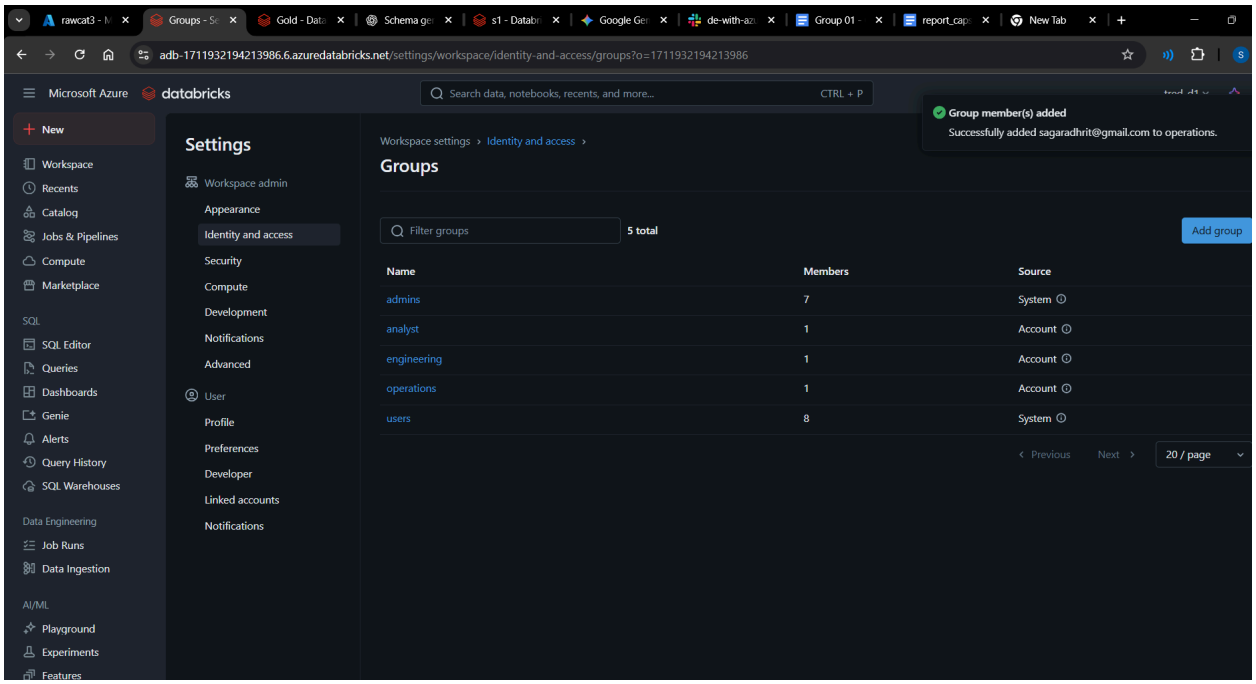


Roles and permissions

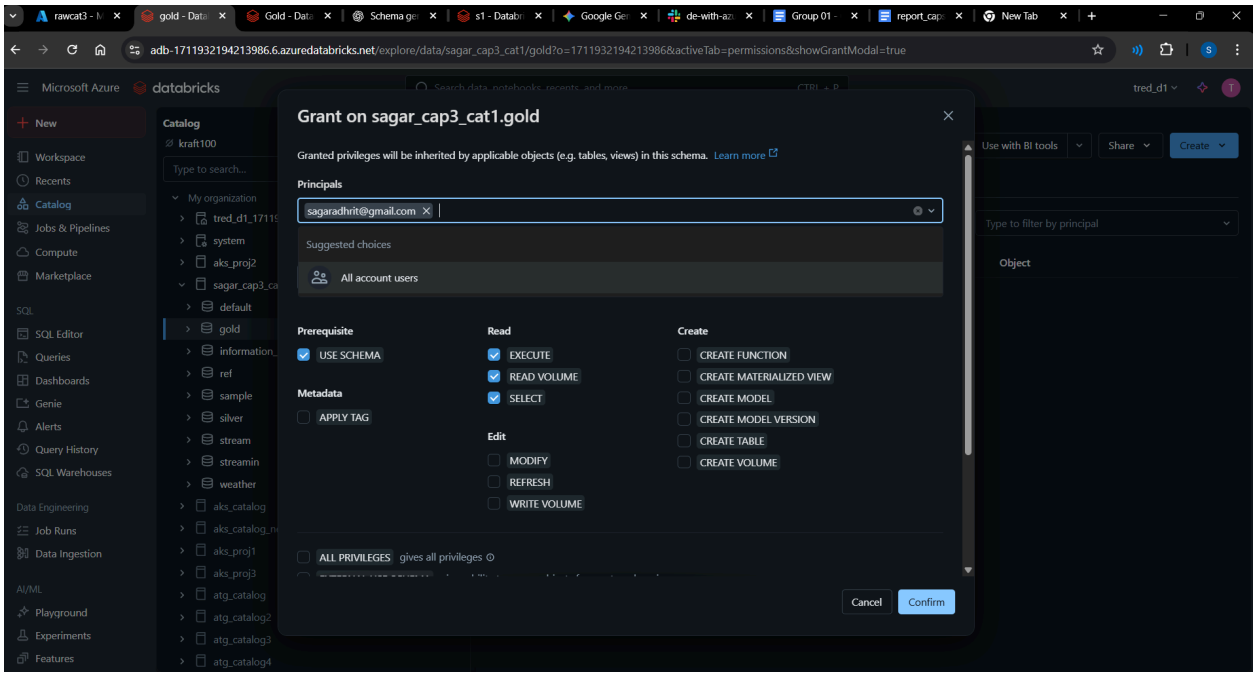
We have implemented a **role-based access control (RBAC)** model to ensure secure and organized access to data. Three groups are defined based on business needs:


adb-1711932194213986.6.azuredatabricks.net/settings/workspace/identity-and-access/groups?o=1711932194213986
Microsoft Azure
databricks
Search data, notebooks, recents, and more...
CTRL + P
tred_d1
New
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie
Alerts
Query History
SQL Warehouses
Data Engineering
Job Runs
Data Ingestion
AI/ML
Playground
Experiments
Features
Settings
Workspace admin
Appearance
Identity and access
Security
Compute
Development
Notifications
Advanced
User
Profile
Preferences
Developer
Linked accounts
Notifications
Workspace settings › Identity and access ›
Groups
Filter groups
5 total
Add group
Name
Members
Source
admins
7
System
analyst
0
Account
engineering
0
Account
operations
0
Account
users
8
System
Previous
Next
20 / page


Group member(s) added
Successfully added sagaradhrit@gmail.com to engineering.
Settings
Workspace admin
Appearance
Identity and access
Security
Compute
Development
Notifications
Advanced
User
Profile
Preferences
Developer
Linked accounts
Notifications
Workspace settings › Identity and access › Groups ›
Group details
engineering
Delete
Group Information
Members
Entitlements
Permissions
Parent groups
Filter group members
Add members
Name
Email/ID
sagaradhrit@gmail.com
sagaradhrit@gmail.com
Previous
Next
20 / page

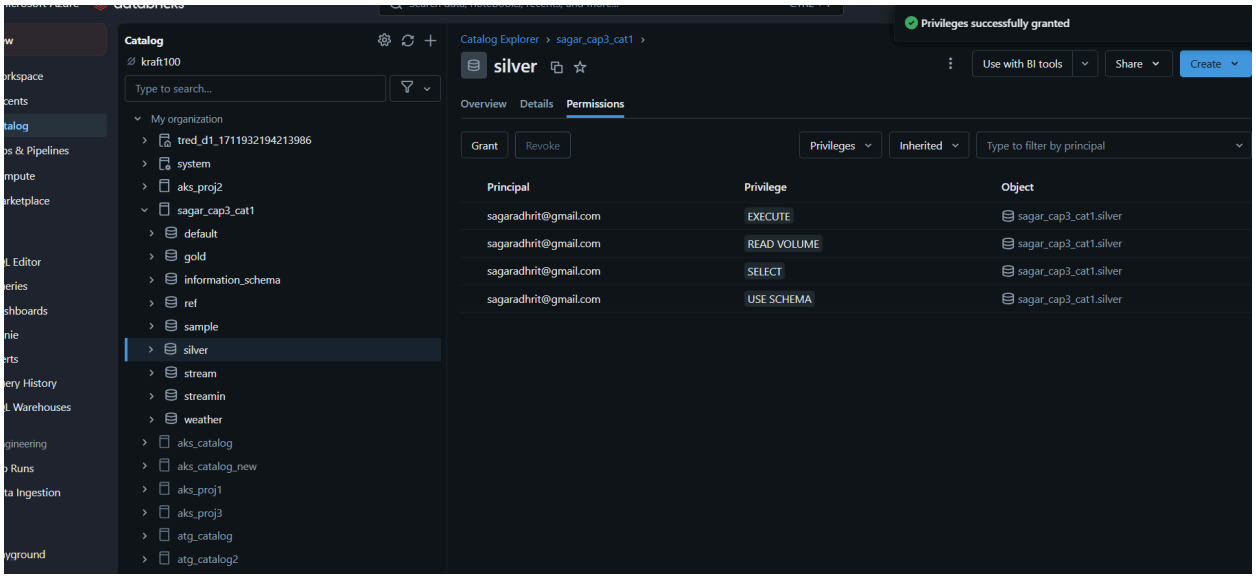


1. **Analyst Group**
    - Members of this group focus on data exploration and reporting.
    - Example: *Sagar* is part of the **Analyst** group.
    - **Access Granted**:
        - **Read-only access** to the entire schema → enables analysts to query and visualize data without altering it.



2. **Operations Group**

   - This group supports day-to-day monitoring and validation of pipelines.
   - Example: *Sagar* also belongs to **Operations**.
   - **Access Granted**:
     - **Read-only access** to the **Silver Layer** → ensures visibility into curated, business-ready data for validations, compliance, and troubleshooting.



3. **Engineering Group**

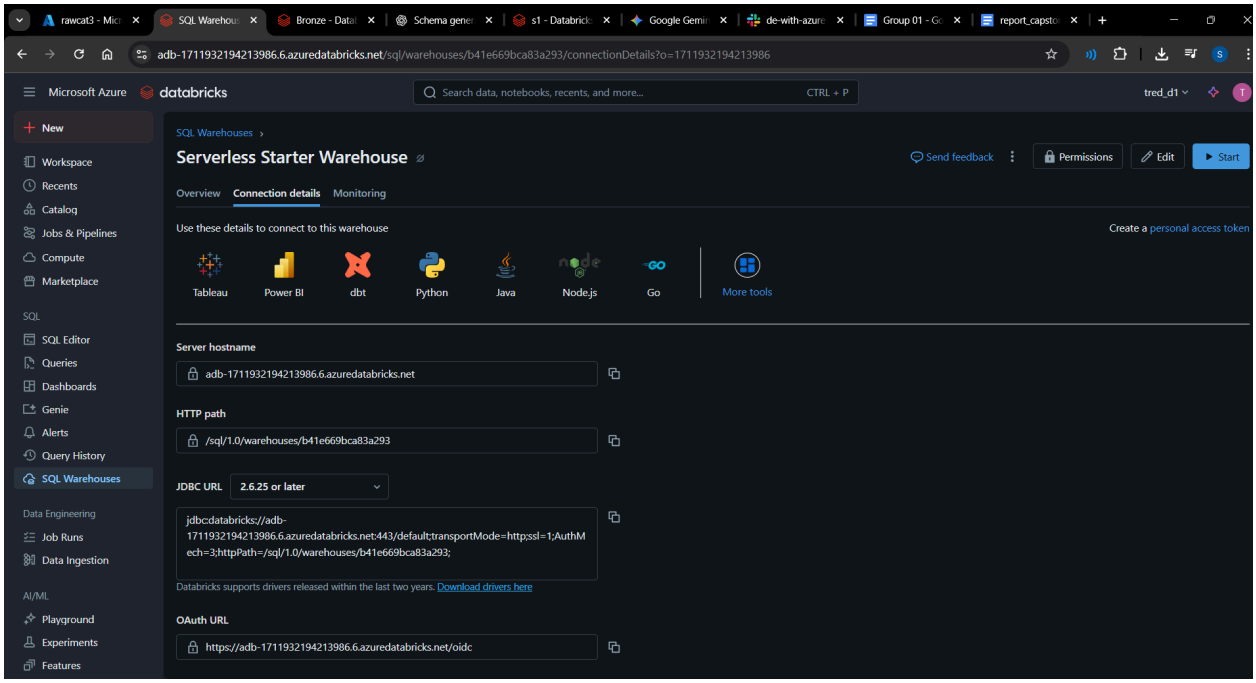
   - Engineers are responsible for building and maintaining pipelines, transformations, and optimization.

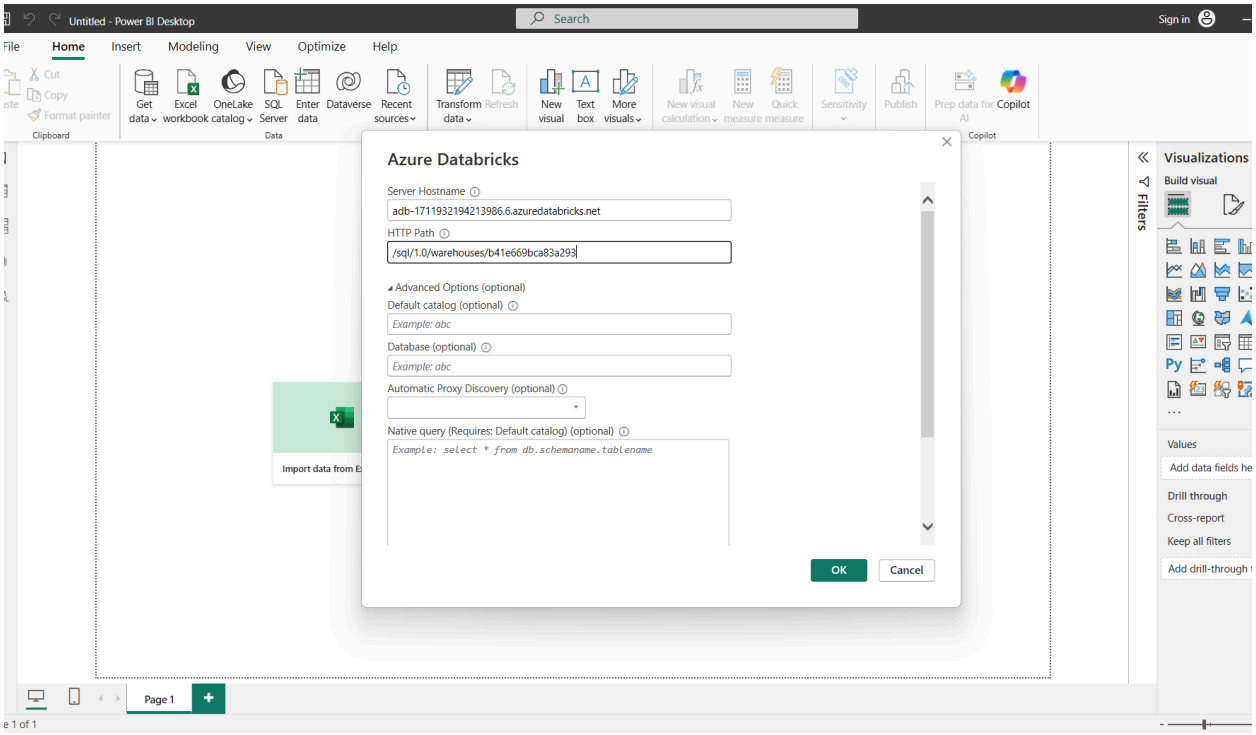
   - **Access Granted**:

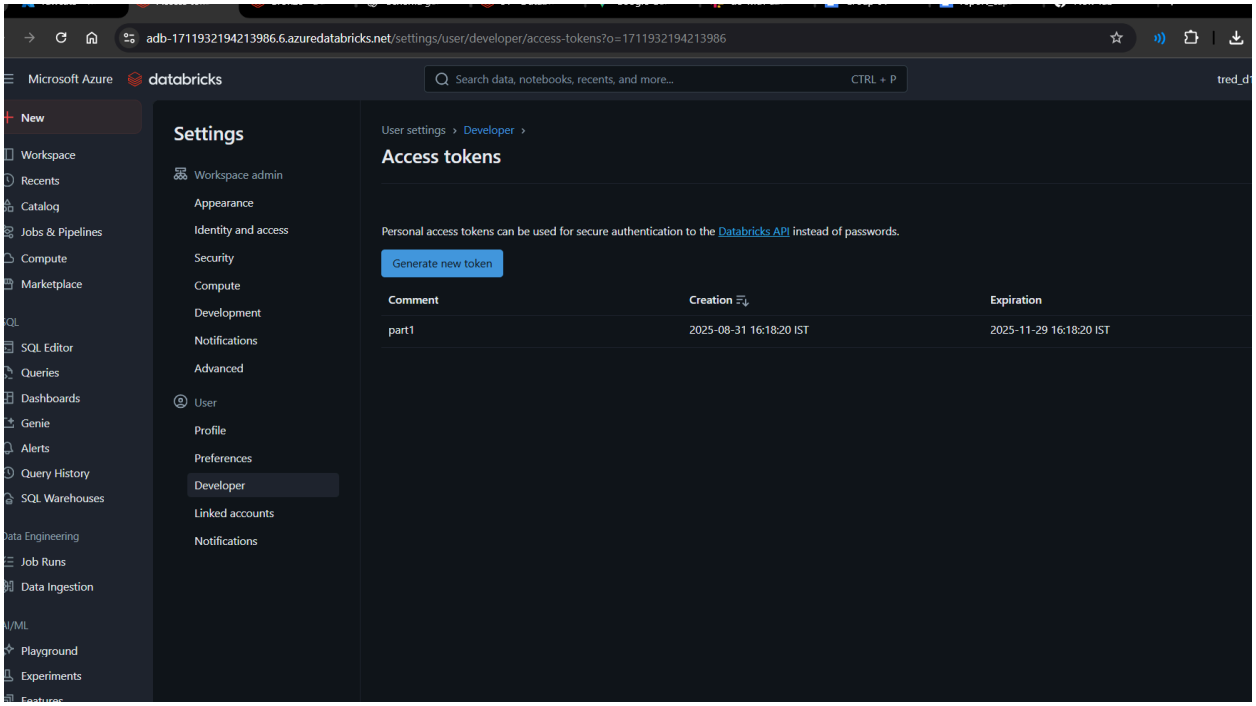
     - **Read and Write access** → to **Silver** layers.

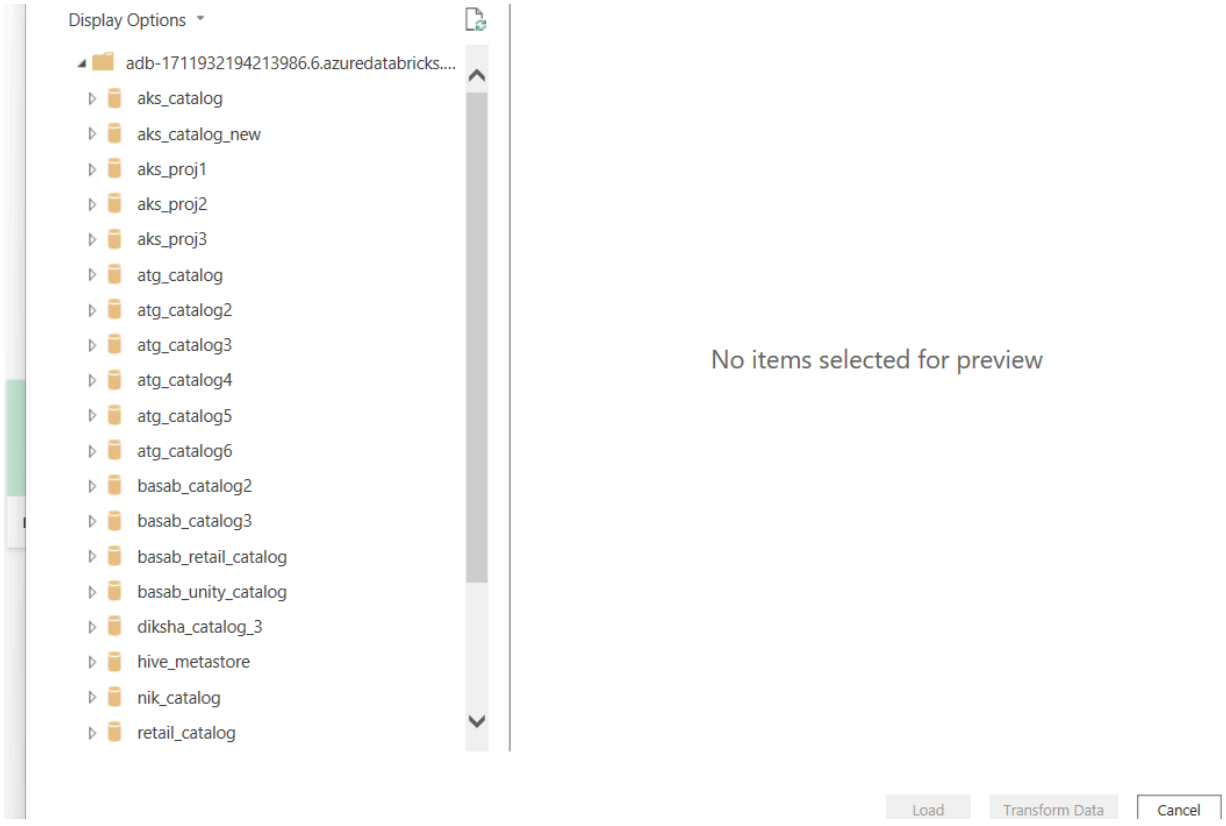
     - Engineers can ingest raw data, apply transformations, and prepare datasets for downstream teams.

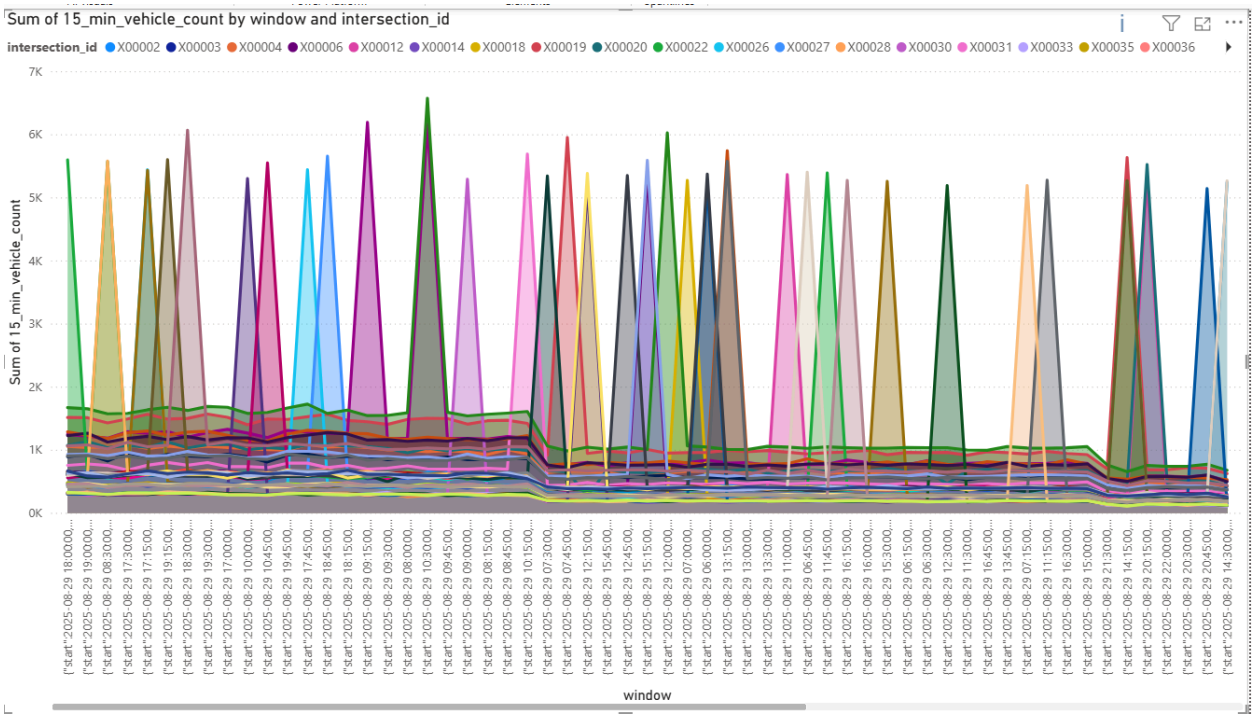
Visualization

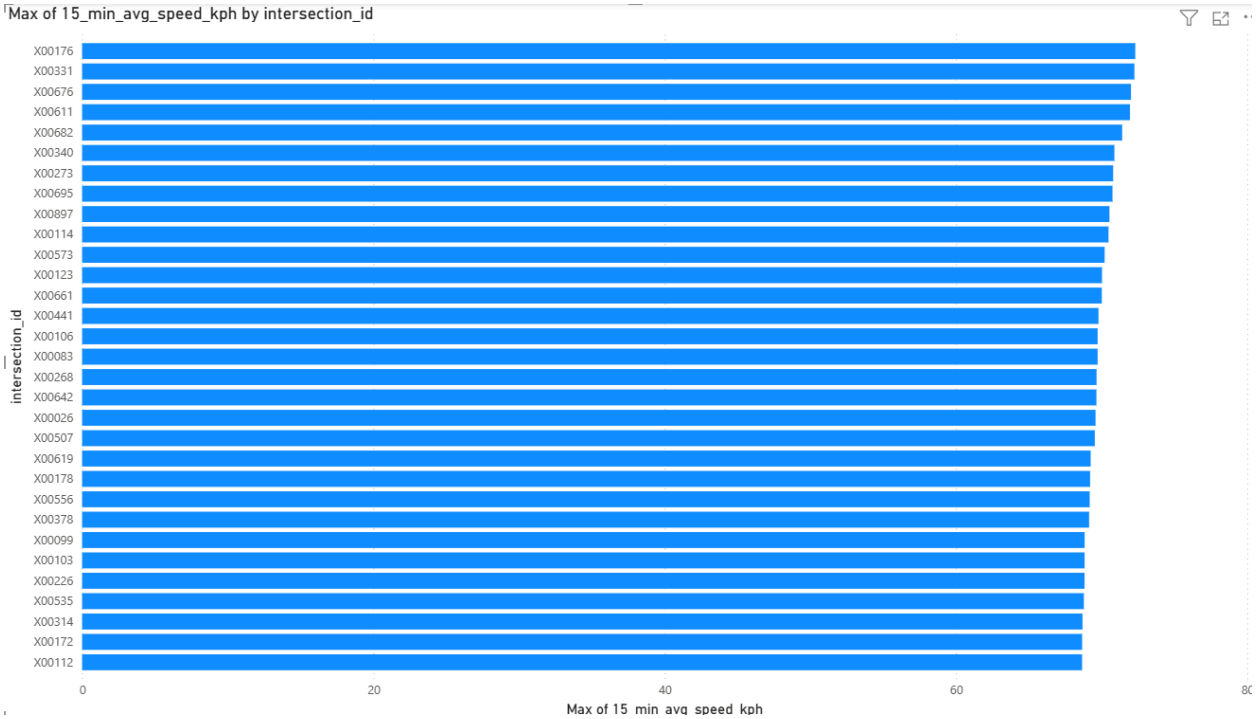




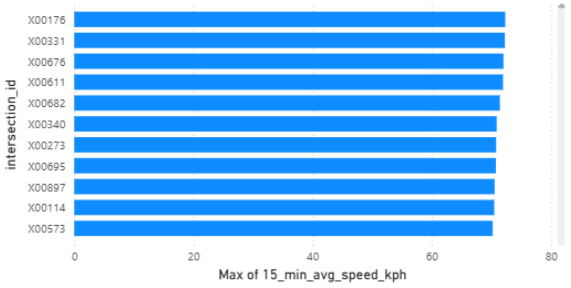

adb-1711932194213986.6.azuredatabricks.net/settings/user/developer/access-tokens?o=1711932194213986
Microsoft Azure
databricks
Search data, notebooks, recents, and more...
CTRL + P
New
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie
Alerts
Query History
SQL Warehouses
Data Engineering
Job Runs
Data Ingestion
AI/ML
Playground
Experiments
Settings
Workspace admin
Appearance
Identity and access
Security
Compute
Development
Notifications
Advanced
User
Profile
Preferences
Developer
Linked accounts
Notifications
User settings › Developer ›
Access tokens
Personal access tokens can be used for secure authentication to the Databricks API instead of passwords.
Generate new token
Comment
Creation
Expiration
part1
2025-08-31 16:18:20 IST
2025-11-29 16:18:20 IST


Display Options
adb-1711932194213986.6.azuredatabricks....
aks_catalog
aks_catalog_new
aks_proj1
aks_proj2
aks_proj3
atg_catalog
atg_catalog2
atg_catalog3
atg_catalog4
atg_catalog5
atg_catalog6
basab_catalog2
basab_catalog3
basab_retail_catalog
basab_unity_catalog
diksha_catalog_3
hive_metastore
nik_catalog
retail_catalog
No items selected for preview
Load
Transform Data
Cancel

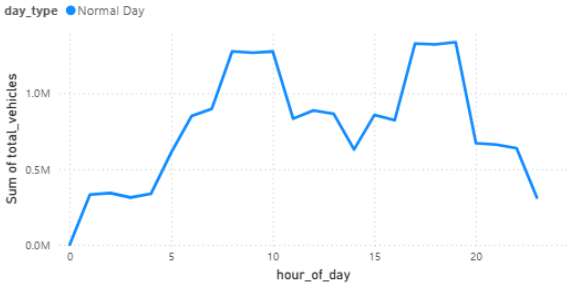

Sum of 15_min_vehicle_count by window and intersection_id
intersection_id X00002 X00003 X00004 X00006 X00012 X00014 X00018 X00019 X00020 X00022 X00026 X00027 X00028 X00030 X00031 X00033 X00035 X00036
Sum of 15_min_vehicle_count
7K
6K
5K
4K
3K
2K
1K
0K
window


Max of 15_min_avg_speed_kph by intersection_id
intersection_id
X00176
X00331
X00676
X00611
X00682
X00340
X00273
X00695
X00897
X00114
X00573
X00123
X00661
X00441
X00106
X00083
X00268
X00642
X00026
X00507
X00619
X00178
X00556
X00378
X00099
X00103
X00226
X00535
X00314
X00172
X00112
0
20
40
60
Max of 15_min_avg_speed_kph

Max of 15_min_avg_speed_kph by intersection_id

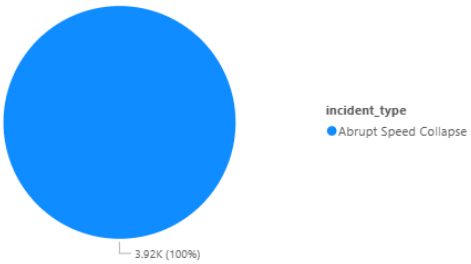


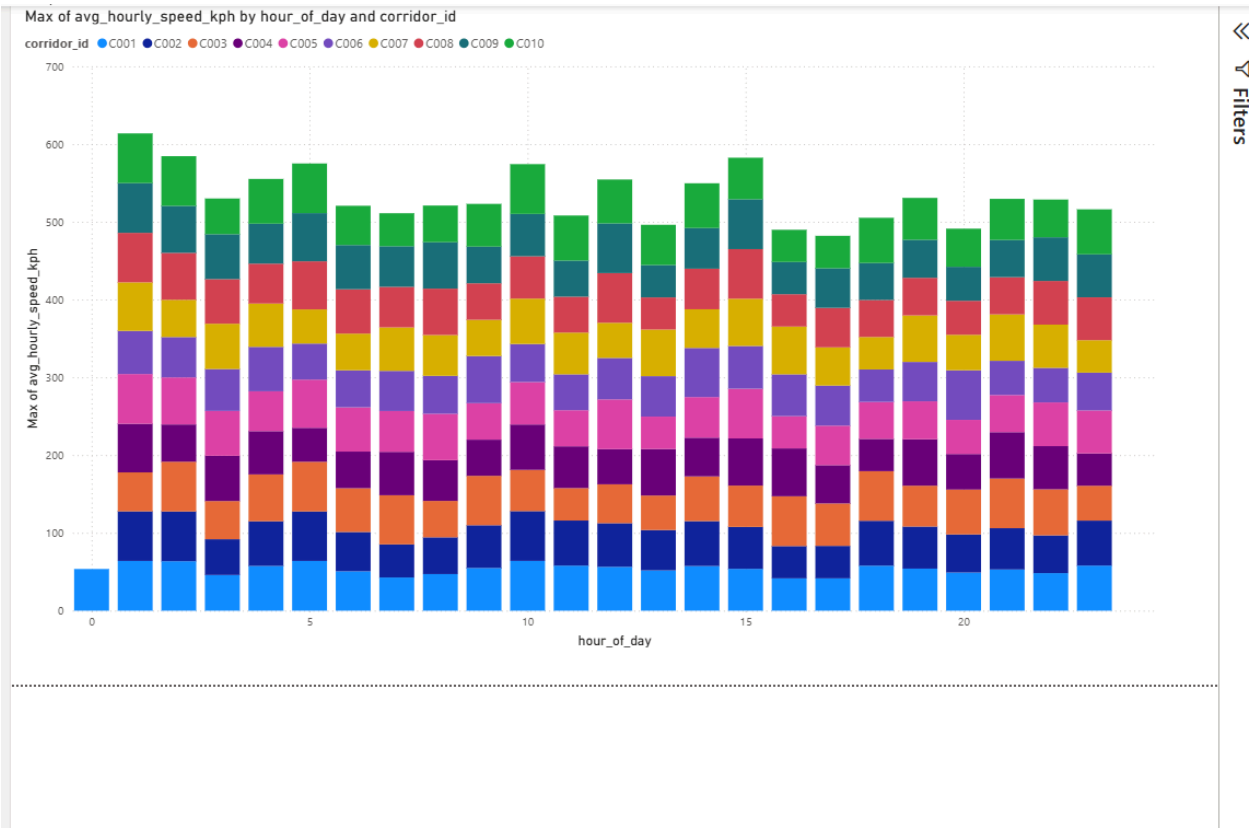
Sum of total_vehicles by hour_of_day and day_type

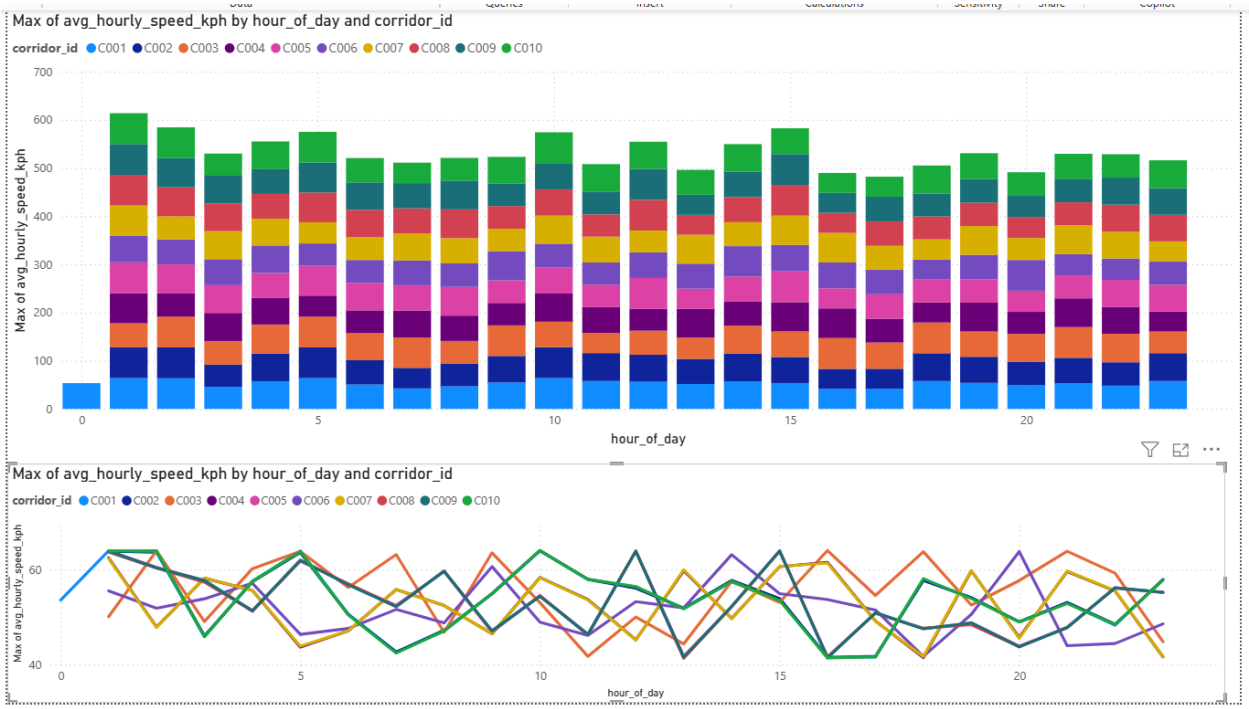


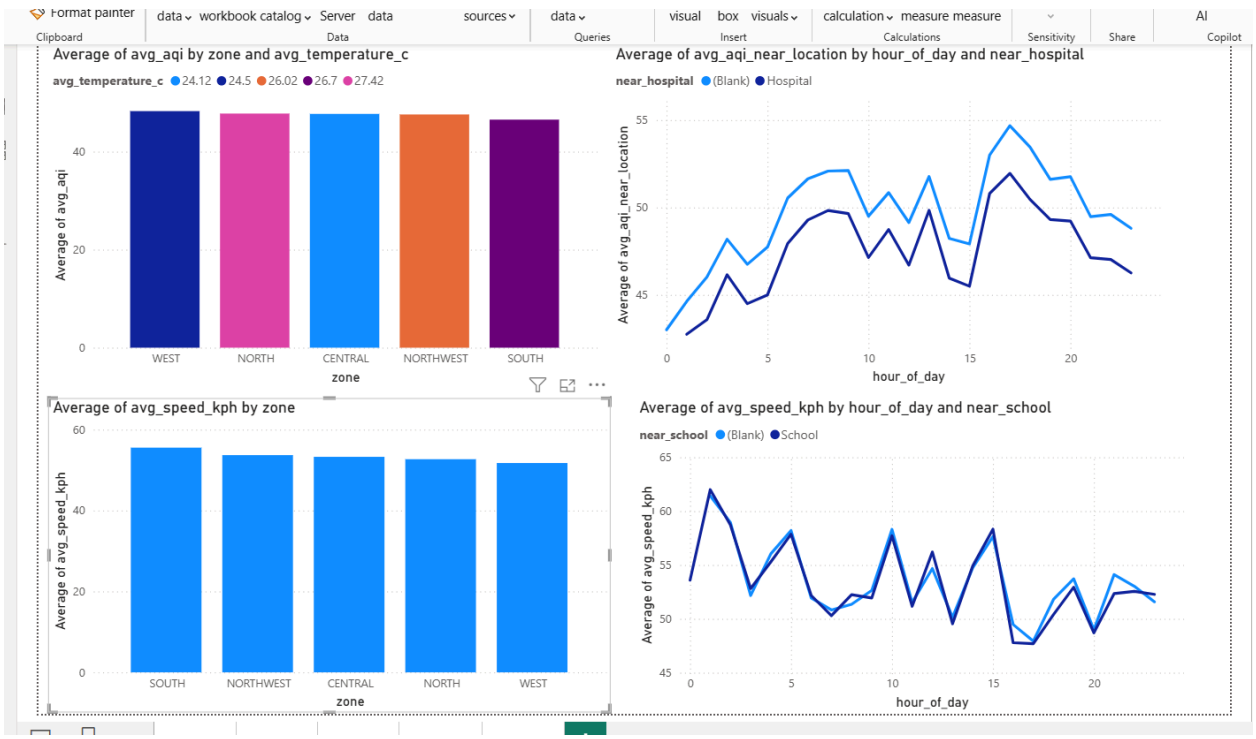
corridor_id	Sum of avg_camera_uptime_pct	Sum of num_camera_faults	Sum of num_firmware_versions
C001	98.60	5	58
C002	97.57	4	49
C003	97.52	3	53
C004	98.32	6	52
C005	98.22	4	49
C006	97.82	6	45
C007	98.51	5	63
C008	98.21	5	47
C009	97.83	9	68
C010	97.36	6	46
Total	**979.96**	**53**	**530**

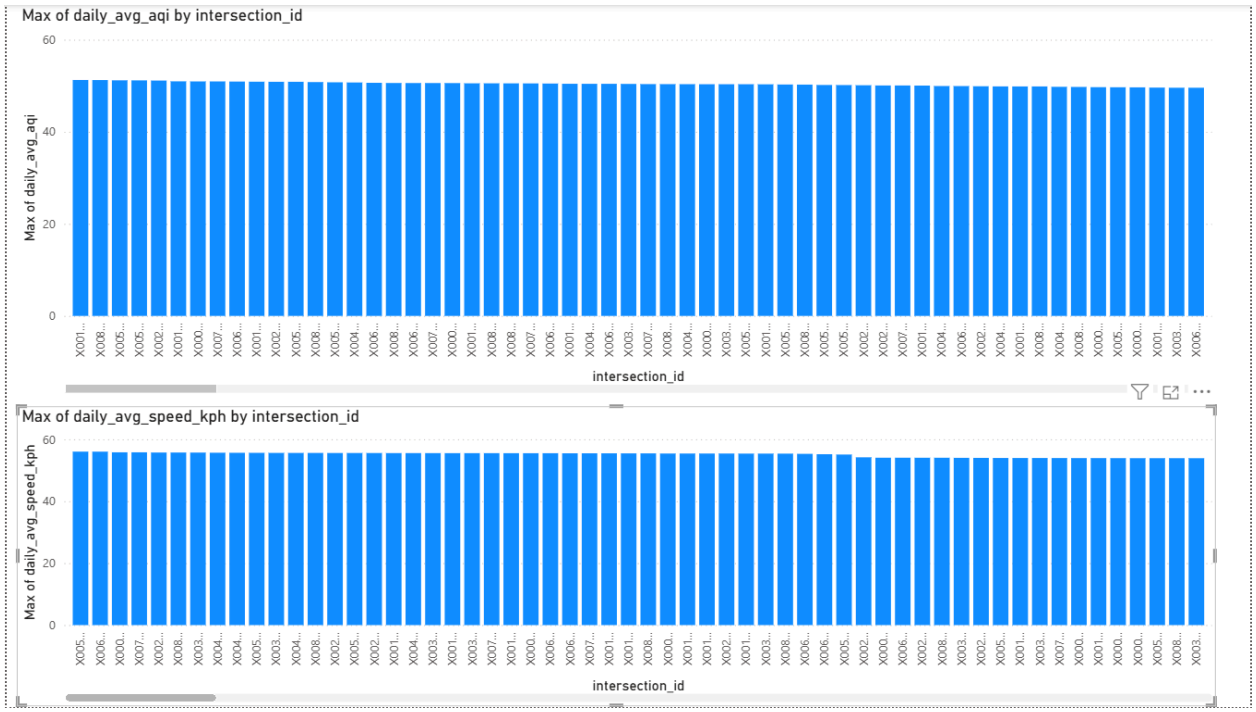
Count of incident_type by incident_type




Max of avg_hourly_speed_kph by hour_of_day and corridor_id
corridor_id C001 C002 C003 C004 C005 C006 C007 C008 C009 C010
Max of avg_hourly_speed_kph
hour_of_day
Filters


Max of avg_hourly_speed_kph by hour_of_day and corridor_id
corridor_id C001 C002 C003 C004 C005 C006 C007 C008 C009 C010
Max of avg_hourly_speed_kph
hour_of_day
Max of avg_hourly_speed_kph by hour_of_day and corridor_id
corridor_id C001 C002 C003 C004 C005 C006 C007 C008 C009 C010
Max of avg_hourly_speed_kph
hour_of_day


Format painter
Clipboard
data
workbook catalog
Server
data
sources
Data
data
Queries
visual
box
visuals
Insert
calculation
measure measure
Calculations
Sensitivity
Share
AI
Copilot
Average of avg_aqi by zone and avg_temperature_c
avg_temperature_c 24.12 24.5 26.02 26.7 27.42
Average of avg_aqi
zone
WEST
NORTH
CENTRAL
NORTHWEST
SOUTH
Average of avg_aqi_near_location by hour_of_day and near_hospital
near_hospital (Blank) Hospital
Average of avg_aqi_near_location
hour_of_day
Average of avg_speed_kph by zone
Average of avg_speed_kph
zone
SOUTH
NORTHWEST
CENTRAL
NORTH
WEST
Average of avg_speed_kph by hour_of_day and near_school
near_school (Blank) School
Average of avg_speed_kph
hour_of_day


Max of daily_avg_aqi by intersection_id
Max of daily_avg_aqi
intersection_id
Max of daily_avg_speed_kph by intersection_id
Max of daily_avg_speed_kph
intersection_id